Live Reconfiguration of Hierarchical Packet Schedulers

Anonymous Author(s)

1 INTRODUCTION

- Programmable packet scheduling. Emphasize that policies are often hierarchical, and that clients demand line rate.
- The reconfiguration problem. Running example: a small-office gateway runs `Strict(hi: gmail, lo: zoom)`, i.e., strictly prioritizing `gmail` traffic over `zoom` traffic. The operator wants to add a new `spotify` flow, and is considering the following two candidates:
  (1) `Strict(hi: gmail, mid: spotify, lo: zoom)`: extend the strict-priority list with `spotify` slotting between `gmail` and `zoom` in priority.
  (2) `Strict(hi: gmail, lo: RoundRobin(zoom, spotify))`: keep `gmail` on top, but have `zoom` and `spotify` share the lower tier via round-robin.

  In either of these cases, the state of the art [1] would stop the world, drop or recirculate buffered packets, recompile, and reinstall. The costs are dropped or recirculated (and therefore delayed) packets, downtime, and the rebuilding of nodes that did not need to be rebuilt.
- The alternative is to reprogram a scheduler without stopping the world. Let's revisit the examples from earlier.
- Transitioning to `Strict(hi: gmail, mid: spotify, lo: zoom)` is actually quite easy. We can achieve the strongest property we could ask for:
  - Before time t_1: `Strict(hi: gmail, lo: zoom)` is running.
  - At time t_1: the request to move to `Strict(hi: gmail, mid: spotify, lo: zoom)` is received. We move to `Strict(hi: gmail, mid: spotify, lo: zoom)`. The transition at t_1 is *atomic*, meaning that whatever user-observable interaction (push/pop) happened immediately before t_1 happened entirely in the `Strict(hi: gmail, lo: zoom)` regime, and whatever push/pop happened immediately after t_1 happened entirely in the `Strict(hi: gmail, mid: spotify, lo: zoom)` regime.
  - This transtion from `Strict(hi: gmail, lo: zoom)` to `Strict(hi: gmail, mid: spotify, lo: zoom)` happens to be both atomic and *immediate*, in that we do not need a period of preparation before we atomically jump into the new policy. The concepts are distinct. [AM note: Can we think of a nice small example that is atomic but not immediate?]
- What about transitioning to `Strict(hi: gmail, lo: RoundRobin(zoom, spotify))`? It is not as easy.
  - Before time t_1: `Strict(hi: gmail, lo: zoom)` is running.
  - At time t_1: the request to move to `Strict(hi: gmail, lo: RoundRobin(zoom, spotify))` is received. We cannot move to the requested policy immediately since there are still `zoom` packets buffered in the scheduler. We atomically step instead into a *transitionary period* that is *not* the user's desired policy[1].
  - After t_1 but before t_2: The scheduler still accepts and emits packets.
  - At time t_2, which is defined to be the instant that a well-defined guard becomes true, we atomically step into the user-requested policy.
- Although the user never specified the semantics of this transitionary period (t_1 to t_2), the period is itself a *de facto* packet scheduling regime: the transition is unavoidable, it persists for nonzero time, and during it the scheduler must continue to field pushes and pops. Something is making scheduling decisions during that window, whether we acknowledge it or not! It is worth recognizing as a scheduling policy in its own right; we call it `link`. Some transitions are better than others from a network operator's point of view, and the state of the art has, perhaps inadvertently, settled on a trivial stop-the-world `link`. Our contributions are to be clear about what `link` is and to improve on it.
- Concretely:
  - Obligation 1. Give a semantics to `link`. The transitionary period is shaped by hardware-level manipulations on the way to realizing `p2`, and not by a clean human-readable semantics. One might naturally worry that it will be hard to pin down the semantics of this transitional regime. We develop a grammar of atomic edits (§3) and a sequencing layer (§4) that carries us from `p1` through one or more `link`s to `p2`. The grammar is carefully designed, such that *any* `link` *produced by that grammar is itself an ordinary scheduling control in the sense of §3.2*: so it *does* have a human-readable semantics, just as user-written policies `p1` and `p2` do.
  - Obligation 2. Improve upon the state of the art. We will give the semantics of the stop-the-world `link` and use it as a baseline. Several practical goals guide us as we look for better ways to transition from a given `p1` to a given `p2`. Can we minimize the length of the transition period? Can we avoid dropped/delayed packets? [AM: more to come here; the cost model is a legit open question!]. We will show that it is always possible to make a transition from any `p1` to any `p2`, but it is occasionally possible to make a very efficient transition. We contribute a tool that achieves this and shows [improvements].

[1] it is `Strict(hi: gmail, lo: Strict*(hi: zoom_pre_t1, lo: RoundRobin(zoom_post_t1, spotify)))`, but understanding why it is precisely that policy is not necessary at present.

2 BACKGROUND & MOTIVATION

2.1 Hierarchical scheduling on hardware

A *PIFO* (push-in, first-out queue) is a priority queue that is additionally defined to break rank-ties in FIFO order. A *PIFO tree* generalizes this to a hierarchy: leaves hold PIFOs of packets, while internal nodes hold PIFOs whose entries are references to the nodes' children. A packet is pushed into some leaf by simultaneously presenting that packet to every node on the path from the root to the leaf; each node enqueues the appropriate entry locally. The best-ranked packet of the tree is popped by popping the root PIFO to obtain a reference to one of the root's children, recursing on that child until arriving at a leaf, and then emitting the packet that the leaf yields. This shape captures hierarchical scheduling: each level expresses one scheduling decision (strict priority, weighted fair queueing, round-robin, etc.), and policies compose via parent-child relationships.

A PIFO tree lowers to hardware by mapping each tree depth onto a dedicated processing element (PE), with siblings and cousins sharing the PE at their depth. Pinning a depth level to a single PE has well-studied benefits for computation and storage; Sivaraman (SIGCOMM '16), Mohan et al. (OOPSLA '23), Shrivastav (SIGCOMM '19), and vPIFO (Zhang et al., SIGCOMM '24) all build on this layout. The PE budget is fixed by the silicon, so the running tree's shape is constrained by what the deployed PEs can host. §3.1 makes this data model precise; the rest of §3 builds the syntactic surface on top.

2.2 vPIFO, and the problem it leaves open

- vPIFO (Zhang et al., SIGCOMM 2024) is the closest related work, and it explicitly leaves our problem open.
- What vPIFO does. It virtualizes a single physical PIFO into many logical "PIFO instances," with a Scheduling Description Language (SDL) and compiler, so their PIFO Visor can *flexibly establish* hierarchical PIFO trees of arbitrary shape on fixed hardware. Its contribution is the reconfigurable substrate.
- What vPIFO does not do. As published, vPIFO has no notion of a difference between old and new policies, no formal semantics for what a policy change means, and no account of in-flight packets during a change. Their SDL IR is for *rank computation* compiled to P4 or CPU, quite different from our structural/topological grammars (our `pol` and δ, §3.2 and §3.3).
- Our two running examples make the gap concrete. To briefly break into terms introduced by vPIFO: the Operation Generation Table and the PIFO Instance Address Table are the structures that encode the running tree's shape and per-instance storage. The easy change, to `Strict(hi: gmail, mid: spotify, lo: zoom)`, looks like the kind of edit that vPIFO's substrate could absorb in place: a targeted append to those tables would plausibly register the new traffic and its operations while leaving the running instances untouched. The harder restructuring, to `Strict(hi: gmail, lo: RoundRobin(zoom, spotify))`, lies beyond what the vPIFO substrate can absorb in place: it changes the running tree's shape (inserting a new internal node `RoundRobin` and re-parenting `zoom` under it), which is not exposed as an in-place edit on either table. The closest vPIFO comes is a full reinitialization onto the new shape, which is precisely the stop-the-world baseline we want to improve on.
- vPIFO names runtime updating as future work, observing that the scheduling of packets during a modification's transitional phase is unresolved.
- The relationship, stated plainly. We are not competing with vPIFO and do not claim a better PIFO substrate. We supply the layer *above* a PIFO substrate (which could well be vPIFO). That is, the formal transition between two policies, the small patch that realizes it, and the transitionary semantics. The two layers compose.

3 A GRAMMAR OF ATOMIC POLICY DIFFERENCES

§3.1 recaps the PIFO tree model. §3.2 defines a small policy DSL and a compiler from the DSL into a hardware-runnable *control*, giving us the syntactic handle on the fully compiled hardware-level scheduler that is actually running. §3.3 fixes a grammar δ of structural edits over that DSL, where every production of δ is, by construction, atomically realizable in hardware. §3.4 proves each production is *sound*: for every production of δ, we say what running it does to the scheduler that is actually running in hardware, show that this matches the policy-level edit we were expecting, and check that the rest of the tree is left alone, with unrelated nodes keeping their state and the live packets staying accounted for. §3.5 argues that this per-production soundness survives the lowering to hardware. The productions of δ need to be arranged in *guarded sequences* in order to realize a full reconfiguration; this is deferred to §4.

3.1 PIFO trees

A *PIFO tree* is a hierarchy of PIFO queues: leaves hold packets, while internal nodes hold child indices that route each `pop` down to a chosen leaf.

Topology vs. contents. A *topology* `t` is a finite tree carrying no data: either a single node `*` or `Node(ts)` which is the parent of a list `ts` of child topologies. A *PIFO tree* of topology `t`, written `q : PIFOTree(t)`, layers data onto `t`. The data is of two forms. A leaf `Leaf(p)` holds a packet-carrying PIFO `p`. An internal node `Internal(qs, p)` itself carries two kinds of data: a list `qs` of well-formed PIFO tree children whose topologies match the corresponding sub-topologies of `t`, and a PIFO `p` whose entries are indices into `qs`. This separation between the topology and the carried contents is key to making the grammar δ of §3.3 well-defined: a structural edit is a change to the topology `t`, distinct from the running contents.

The two observable operations. `push(q, pkt, pt)` enqueues `pkt` into tree `q` along a precomputed path `pt` $= (i_1, r_1) :: \ldots :: (i_n, r_n) :: r_{n+1}$. The path is richly decorated: it tells the PIFO of each internal node along the path what child index to enqueue and what rank to use for that enqueue. The trailing element is asymmetric: a bare rank r_{n+1} with no slot index, since it tells the

leaf's PIFO what rank to use when enqueuing the packet itself. pop(q) returns the most favorably ranked packet in the tree by popping the tree's root to yield a child index, recursing into that child, until finally emitting a packet from the leaf. These are the *only* observable interactions with a scheduler, which is why our notion of an *atomic* transition is stated in terms of push/pop observability.

Well-formedness. A PIFO tree q is well-formed (written ⊢ q) when, at every internal node with index-PIFO p and children qs, the number of occurrences of i in p equals the number of packets held under qs[i], for every legal i. This is the invariant that keeps pop from ever getting stuck. push always preserves ⊢ q, and pop preserves it when q is non-empty (which is precisely the condition under which pop is defined).

[AM note: this is still stated in the big brain model. If we want to state it in the small-brain model, I can, but that is not exactly faithful to Mohan et al (see line just below).]

We adopt this much from Mohan et al. [2]; the rest of the formalism in this paper is ours, and we preserve backward compatibility with theirs.

3.2 A Policy DSL

The formalism we have inherited from §3.1 is a runtime model: a PIFO tree is the object that runs in the scheduler, with no source-level surface for an operator to express a desired policy. To talk about reconfigurations, we need to expose a programming interface for the network operator. We design a small policy DSL pol using which the operator can specify their desired policy, and a compiler from pol terms to runnable PIFO tree *controls* (the per-node triples defined below). The transition planner of §4 needs both a way to compile a starting control C from an operator's request and a way to compare two user requests to identify how they differ. pol is the common syntactic surface both rely on.

This is essentially what the vPIFO paper's *Scheduling Description Language* does informally [1]. They do not pin down a grammar for SDL or formalize the compilation, so our DSL can be read as a formal core of their concrete language. The compilation targets differ: vPIFO compiles straight to a virtualized PIFO substrate, whereas we compile first to a control (the abstraction that this section reasons over) and only then lower to a substrate (§6). The strategy, though, is the same: give the operator a syntactic surface, then compile.

Policy syntax: pol.

```
pol    ::= flow
         | D(pol_1, ..., pol_n)
```

A pol is either a leaf labeled by a flow or an internal node running discipline D over n child policies. D ranges over the disciplines (Strict, RoundRobin, WFQ, etc.). A discipline may need per-arm metadata that shapes how the arm is scheduled. WFQ takes a positive real weight per arm; Strict takes a priority rank per arm; RoundRobin takes nothing. The grammar carries no structural mark of any of this. The operator writes the metadata in the surface syntax (e.g., WFQ(w_1: pol_1, ..., w_n: pol_n), or positional sugar like Strict(A, B) which desugars to Strict(hi: A, lo: B)). Once compiled, the metadata lives in the arm's slot_state (see init_slot_D below), where push/pop can read it but not modify it. We read the arity off by counting children, so Strict(gmail, zoom) is the 2-ary instance of the discipline Strict. Each leaf label denotes a *flow*: a predicate over packets.

Arm order in the surface notation is a presentation choice, not a scheduling-meaningful one: Strict(hi: A, lo: B) and Strict(lo: B, hi: A) describe the same scheduler. We formalize this below as the *reorder-congruence* $=_R$ on pol. The compiler has a degree of freedom here, in that it may pick any representative of an $=_R$-class when laying out or editing a control. For instance, a compiler might run a deterministic normalize operation, sorting Strict arms by rank, RR arms by content, and so on, then commit to that one representative throughout. Doing so reduces edit footprints: a reconfiguration that only permutes siblings becomes a no-op (both endpoints normalize to the same pol), and one that permutes siblings while also adding an arm is viewed as "just an arm addition". The framework leaves the choice to the implementation; any strategy that respects $=_R$ is fine, and our implementation (§6) follows the normalize-and-commit-to-it strategy just sketched.

A pol is written against a fixed *flow universe* F: a finite set of flow predicates declared by the operator before constructing the pol. A pol is *valid* over F when (a) every discipline is applied at the proper arity, (b) every discipline is provided with the per-arm metadata that the discipline requires, and (c) the leaves form a partition of a subset of F: each leaf carries a flow drawn from F, and no two leaves carry the same flow. Any packet not matching any leaf-used flow is dropped. Validity is a condition on the source pol, not to be confused with the runtime invariant ⊢ q.

Discipline compilation: init_node_D *and* init_slot_D. Each discipline D comes with a mechanical recipe for compiling a node that runs it: (a) a per-node scheduling transaction, (b) an initial node_state for the node, and (c) a slot_state for each child of the node. We name the two state-seeding projections:

```
init_node_D : () -> node_state
init_slot_D : node_state x meta? -> slot_state
meta?  ::= eps  |  priority-rank  |  weight
```

The meta? argument is the per-arm metadata that discipline D requires: a weight for WFQ, a priority rank for Strict, absent for RoundRobin. init_node_D is called only at *compile time*, once per node, to seed that node's node_state. init_slot_D is called in two places. At *compile time*, walking the source pol, it is called once per child arm to seed that child's slot_state, taking the parent's just-seeded node_state and the arm's meta? as input. When a new arm is spliced under an already-running D-parent (§3.4.1), init_slot_D is called once with the parent's *current* node_state and the new arm's meta? to produce the new arm's slot_state.

Choosing init_slot_D is a scheduling decision, not just a structural one, since the choice changes how a freshly spliced arm competes with the established arms. We broadly make the choice to "join the current round". For example, if going from WFQ(A,B) to WFQ(A,B,C), we do not want the newly added C to reap a huge benefit for "having been silent all this while"; we just want it to join the others with neither a penalty nor an advantage. The choice plays out as follows for each D.

- **RoundRobin.** `init_slot_RoundRobin` returns the empty tuple; RR has no per-arm bookkeeping. The round-robin cursor lives in `node_state` instead, and at `Add` we splice the new arm into the child list at the cursor's current position, so its first turn lands in the round already in progress. The cursor itself is left pointing at the same physical arm it was pointing at before the splice.
- **Strict.** `init_slot_Strict(_, p) = p`: the arm's `slot_state` is just its priority rank, drawn from a dense total order such as the rationals so that a fresh priority can always be slotted strictly between two existing ones.
- **WFQ.** The `node_state` at a parent is the virtual time `vt`, and `init_slot_WFQ(vt, w) = (w, vt)`: a new arm carries its weight `w` and inherits the parent's current `vt` as its last-finish tag. Any standard WFQ tagging that derives the next packet's tag from the slot's last-finish tag and the parent's `vt` (e.g., the GPS-style `max(virtual_clock, vt) + 1/w`) then slots the first packet on this arm into the round that the established arms are currently in. At compile time the parent's `vt` is freshly initialized (to zero), so all original arms get `(w, 0)` and the round is "the zeroth"; at `Add` the parent's `vt` is whatever the clock has advanced to.

Policy Compilation: `pol` *to control.* A PIFO tree *control* `C` is a tree of triples `(state, pifo, z)`, one per node of the topology. The tree shape exactly matches that of `pol`, as each node of `C` lines up with a node of the source `pol`.

We write $\lceil p \rceil$ for the control compiled from `p`; the compile rule fills in the local triple at each node of `p`'s topology as follows:

- `state` is a pair `(node_state, slot_state list)`. The `node_state` is seeded by `init_node_D()`. The `slot_state` list carries per-arm bookkeeping, each entry seeded by `init_slot_D`. When the node runs a discipline without per-arm bookkeeping (e.g., `RR`), it has an empty `slot_state` list.
- `pifo` is an empty PIFO: an index-PIFO at an internal node, a packet-PIFO at a leaf.
- `z` is `D`'s *scheduling transaction* at the node. It examines the local `state` and the incoming packet and produces a path segment and an updated `state`. We write $\rightharpoonup$ for partial functions; `z` is partial because not every packet is admitted at every node. The shape of the path segment differs between internal nodes and leaves:
  - at an internal node, `z` : `state`×`Pkt` $\rightharpoonup$ (`idx`×`rank`)× `state`: pick a child index `i` and the rank `r` with which to enqueue `i` at this node's index-PIFO;
  - at a leaf, `z` : `state` × `Pkt` $\rightharpoonup$ `rank` × `state`: pick the rank `r` for the packet's own PIFO entry.

  When `z` is undefined for a packet, nothing is enqueued at this node and `state` is unchanged. It is important for well-formedness that, when `z` is defined (resp. undefined) for a packet, it is defined (resp. undefined) along the entire path from leaf to root. This global property is not a concern of node-local `z`s.

We address node-local components of a control by a `path`, a (possibly empty) sequence of child indices read from the root. The local triple at the node reached by following `path` from control `C`'s root is written `C@path`, with fields `C@path.state`, `C@path.pifo`, `C@path.z`. We also write `C@path.node_state` and `C@path.slot_states` for the two components of `C@path.state`.

Well-formedness: $\vdash$ `C`. In §3.1 we defined well-formedness on a PIFO tree, written $\vdash$ `q`. Now we redefine it, lifting it to act on controls. A control `C` is *well-formed* (written $\vdash$ `C`) when, at every internal node of `C`, the `pifo` has, for each legal child index `i`, exactly as many occurrences of `i` as there are packets stored in the leaf pifos of the subtree under the `i`-th child. This is the same well-formedness property as before, and maintaining it has the same effect (preventing `pop`s from getting stuck). We just state it directly on `C` so that no global PIFO tree needs to be assembled to check it.

Backward compatibility. A control of the kind we have just defined is sometimes packaged instead as a single *monolithic triple* `(s, q, z)`: a state map `s` indexed by path, a single global PIFO tree `q`, and a single global scheduling transaction `z` : `St` × `Pkt` $\rightarrow$ `Path(t)` × `St`. Our distributed controls flatten into such a triple by gluing the per-node pieces: the tree of our `pifo`s assembles into `q`, the `state`s indexed by path assemble into `s`, and a top-down composition of the per-node `z`s yields the global `z`, with partiality preserved (a drop anywhere along the descent leaves the global function undefined for that packet). The rest of the paper has no need for this gluing, since $\vdash$ `C` is stated directly above and the per-production rules of §3.4 act node-locally; we record it so that any apparatus stated against the monolithic shape can still be applied on top of ours.

Equivalences: $\sim$, $=_R$, $\sim_R$. We need three equivalence relations: one on controls under live operation ($\sim$), one on `pol`s under sibling permutation ($=_R$), and one on controls under live operation *and* sibling permutation ($\sim_R$).

`push` and `pop` change the live `pifo`s and `state`s of a control but leave its structural skeleton untouched. We write `C` $\sim$ `C′` for the equivalence relation on well-formed controls that identifies any two controls related by a finite sequence of `push` / `pop` operations.

We write $p =_R p'$ for the smallest congruence on `pol` such that, at any internal `D`-node, permuting the child arms gives a congruent pol: $D(p_a, \ldots, p_z) =_R D(p_{\sigma(a)}, \ldots, p_{\sigma(z)})$ for any permutation σ. The per-arm metadata that discipline `D` requires (a `WFQ` weight, a `Strict` priority rank) travels with its arm under the permutation; the metadata is what carries the scheduling-meaningful content, so a permutation does not change the scheduler.

Two controls whose child lists at some internal node are permutations of one another (with the parent's `pifo` and `z` renumbered accordingly) present different positional layouts but realize the same scheduler. We write `C` $\sim_R$ `C′` for the equivalence obtained by closing $\sim$ under such sibling permutations. Every $\sim$-equivalent pair is $\sim_R$-equivalent; the converse fails.

The bridge: $\lfloor \cdot \rfloor$. We write $\lfloor C \rfloor$ to mean "the `pol` that `C` realizes". $\lfloor \cdot \rfloor$ is pinned down by three rules:

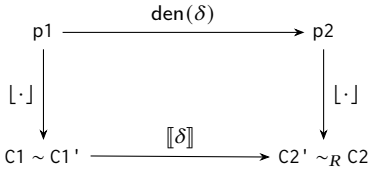
(1) *Base case (compilation).* $\lfloor \lceil p \rceil \rfloor =_R p$. The compiler is free to pick any sibling order when laying `p` out as a control (creating $\lceil p \rceil$), and then we read off the exact `pol` that $\lceil p \rceil$ realizes using $\lfloor \lceil p \rceil \rfloor$. This is not exactly equal to `p`, but we can use the flexibility that $=_R$ affords us to relate them.

(2) *Closure under pushes and pops.* If $\texttt{C} \sim \texttt{C}'$, then $\lfloor C \rfloor = \lfloor C' \rfloor$. Pushes and pops touch only live `state` and `pifo` contents; they leave the topology and `z` of every node verbatim, so the pol-level skeleton that $\lfloor \cdot \rfloor$ names is untouched.

(3) *Closure under δ.* Each δ has two readings, both defined per-production in §3.4: an operational rewrite on the live control, written $[\![\delta]\!] : \texttt{control} \rightharpoonup \texttt{control}$, and a closed-form pol-level effect, written $\mathsf{den}(\delta) : \texttt{pol} \rightharpoonup \texttt{pol}$. §3.3 fixes the grammar; for now, we just assert that every δ we will define both readings. Rule 3 says the two readings agree: $\lfloor [\![\delta]\!](C) \rfloor =_R \mathsf{den}(\delta)(\lfloor C \rfloor)$. The $=_R$ slack is needed here because the operational rewrite has arm-order freedom, just as the compiler does.

The three rules together let us propagate $\lfloor \cdot \rfloor$ from any $\lceil p \rceil$ along any sequence of pushes, pops, and productions of δ. This is how we will discharge Obligation 1 of §1: telling the operator what `pol` is running even when no user has explicitly requested the `pol`.

The interplay of the three rules is captured by the following diagram.



Let us study this diagram with an eye to the user's experience. Our final goal will be to correctly relate `C2'` and `C2`.

- The operator writes `p1`. Then $\texttt{C1} := \lceil \texttt{p1} \rceil$, with $\lfloor \texttt{C1} \rfloor =_R \texttt{p1}$ by rule 1.
- We echo $\texttt{p1}' := \lfloor \texttt{C1} \rfloor$ back to the user. This is not shown in the diagram but will become important shortly. `p1'` faithfully represents the arm ordering that the compiler chose.
- Push and pop operations carry `C1` to `C1'`. $\texttt{C1} \sim \texttt{C1}'$ by the definition of $\sim$, and $\lfloor \texttt{C1}' \rfloor = \lfloor \texttt{C1} \rfloor$ by rule 2, so the live `C1'` still realizes both `p1` and `p1'`.
- The operator writes `p2`. The *transition planner* (§4-§5), the tool that infers atomic edits between two `pol`s, produces a δ such that $\mathsf{den}(\delta)(\texttt{p1}') =_R \texttt{p2}$. It is key that we work in the frame of the actually-running representative `p1'` rather than the operator's original `p1`, since `den` is stated using semantically meaningful paths.
- Applying $[\![\delta]\!]$ to `C1'` brings us to control `C2'`, and by rule 3 $\lfloor \texttt{C2}' \rfloor =_R \mathsf{den}(\delta)(\texttt{p1}')$. Further, we can chain this with the fact $\mathsf{den}(\delta)(\texttt{p1}') =_R \texttt{p2}$ (established just above) to get $\lfloor \texttt{C2}' \rfloor =_R \texttt{p2}$.
- We again echo $\texttt{p2}' := \lfloor \texttt{C2}' \rfloor$ to the user.

The transformation is complete at this point, but we need to ground ourselves. $\texttt{C2} := \lceil \texttt{p2} \rceil$ is the control we *would have built* had we taken the SOTA stop-the-world path: the state-of-the-art response to a $\texttt{p1} \rightarrow \texttt{p2}$ request is to freshly compile $\lceil \texttt{p2} \rceil$ (§1). We do not actually construct `C2`, but it is the correct reference point and it is crucial that we now relate `C2'` (which we have just produced after a fashion) to `C2`. By rule 1, $\lfloor \texttt{C2} \rfloor =_R \texttt{p2}$, hence $\lfloor \texttt{C2}' \rfloor =_R \lfloor \texttt{C2} \rfloor$. But we would like to relate the controls directly, not just their `pol`-level projections. The relation we write is $\texttt{C2}' \sim_R \texttt{C2}$, which absorbs two gaps at once:

- `C2'` carries the live `pifo`/`state` accumulated since `C1`, while `C2` is freshly compiled and bare. The $\sim$ component covers this.
- $[\![\delta]\!]$ and $\lceil \cdot \rceil$ are free to pick different sibling orders at internal nodes, so `C2'` and `C2` may also differ in child arrangements. The R-closure covers this.

The diagram above is the unit cell of a fuller picture. A typical reconfiguration is not a single δ but a guarded sequence $(\varphi_0; \delta_0); \ldots; (\varphi_n; \delta_n)$; §4.1 stacks copies of this unit cell horizontally, with each intermediate control $\texttt{link}_i$ between `C` and `C'` carrying its own pol-level meaning $\lfloor \texttt{link}_i \rfloor$. For §3.2's purposes we read the unit cell on its own; everything that follows about a single δ, including the operator-advice below, lifts cell-by-cell to the sequence picture in §4.1.

When writing `p2`, the operator would do well to state their request against `p1'`, the actually-running `pol` that we echoed back to them. This keeps δ small and aligns paths with what is running. The operator may state `p2` against the `p1` they originally wrote, but at their own risk. The risks are of two flavors:

- The planner may infer a larger δ than necessary, or may give up.
- The more serious issue is that the user may use Imperative Mode (§4) to directly specify what edits to make, and if they use `p1` to base their paths, they may inadvertently edit the wrong node or provide a malformed path.

A worked example.

- The operator requests `p1 = Strict(hi: gmail, lo: zoom)`.
- The compiler uses its degree of freedom to yield control `C1` such that $\lfloor \texttt{C1} \rfloor = \texttt{Strict}(\texttt{lo} : \texttt{zoom}, \texttt{hi} : \texttt{gmail})$. Note that the children have been reordered for some compiler-internal reason.
- We echo back

  $$\texttt{p1}' := \lfloor \texttt{C1} \rfloor = \texttt{Strict}(\texttt{lo} : \texttt{zoom}, \texttt{hi} : \texttt{gmail}).$$

  We know that $\texttt{p1} =_R \texttt{p1}'$, so the scheduler the operator gets is the one they asked for; only the slot numbering differs.
- As the control serves pushes and pops, it transforms into `C1'`.
- Later, the operator requests `p2 = Strict(lo: zoom, mid: spotify, hi: gmail)`.
- The runtime can again use its freedom. Instead of literally splicing `spotify` in between running arms `zoom` and `gmail`, it chooses to append `spotify` to the end. This converts the running control `C1'` into control `C2'` such that

  $$\lfloor \texttt{C2}' \rfloor = \texttt{Strict}(\texttt{lo} : \texttt{zoom}, \texttt{hi} : \texttt{gmail}, \texttt{mid} : \texttt{spotify}).$$

- We echo back

  $$\texttt{p2}' := \lfloor \texttt{C2}' \rfloor = \texttt{Strict}(\texttt{lo} : \texttt{zoom}, \texttt{hi} : \texttt{gmail}, \texttt{mid} : \texttt{spotify})$$

  to the user.

- Now the operator changes to Imperative Mode (§4) and writes the path-bearing edit `(True, Quiesce([2]))`. It is not worth getting distracted by the syntax or the semantics; the key thing is that the operator has requested an edit and has identified the target via path "`[2]`". Paths are interpreted against the *actually running* representative `p2'`, so we `Quiesce` the subtree `mid: spotify` (`p2''`s third slot), not `lo: zoom` (`p2`'s third slot). If the operator had based the path on `p2` they would have edited the wrong arm; the system has no way to detect or recover from that.

3.3 A Grammar for Tree Differences

We fix a small grammar δ of atomic edits (we use "atomic" informally here; §3.4 pins it down formally). Each production of the grammar has two readings, both defined per-production in §3.4: an operational rewrite $[\![\delta]\!]$: `control` ⇀ `control`, and a closed-form pol-level effect $\mathsf{den}(\delta)$: `pol` ⇀ `pol`. The grammar is shaped by two demands.

- **Hardware-realizable.** A production is admitted into the grammar iff $[\![\delta]\!]$ can be committed atomically by the hardware substrate (§6).
- **Pol-explainable.** Every production has a $\mathsf{den}(\delta)$[2]. This is what lets the planner (§4) infer a δ whose `pol`-level meaning can be checked and echoed back to the user.

§3.4's per-production soundness theorem ties the two readings together: $\lfloor [\![\delta]\!](C) \rfloor =_R \mathsf{den}(\delta)(\lfloor C \rfloor)$.

Each edit names *where* in the tree the change lands and *what* the change is. Throughout this section, `p1` is the presently running policy; we write `p1@path` for the subtree of `p1` reached by following `path` down from its root. One production is irregular in this respect: `ChangeRoot` gives the path to the surviving subtree rather than to the edit site. We explain this in §3.4.

```
d ::= Add          (path, arm, meta?)
    | ChangeMeta   (path, meta)
    | Quiesce      (path)
    | Remove       (path)
    | Designate    (path, survivor)
    | Undesignate  (path)
    | ChangeRoot   (path)

path   ::= []  |  i :: path
```

A `path` is a (possibly empty) sequence of child indices `i` read from the root. `pol` is the nonterminal of §3.2; `arm` and `survivor` are field names for `pol`-typed payloads, chosen to read at the call site (the new arm under an `Add`, the designated survivor under a `Designate`).

Edits that would have to destroy structure still holding packets are expressly not in the grammar. For example, structural deletion, `Remove`, is emitted by our transition planner (§4) only after ensuring that the subtree being removed is empty. The richer reconfigurations an operator may want (retiring a subtree that has packets buffered in it, replacing a subtree in-place, pruning a tree down to a subtree) are realized as *sequences* of these productions (§4).

[2]for transaction-only productions (whose effect lives entirely in `z`), $\mathsf{den}(\delta)$ is the identity on `pol`

Brief notes on each production:

- `Add(path, arm, meta?)` makes `arm` a new child of `p1@path`. `meta?` carries per-arm bookkeeping for the new arm, if `p1@path` requires it.
- `ChangeMeta(path, meta)` overwrites the per-arm metadata assigned to `p1@path`, interpreted per the parent's discipline: a priority rank under Strict, a weight under WFQ.
- `Quiesce(path)` prevents `p1@path` from receiving any new traffic.
- `Remove(path)` removes `p1@path`, which must be empty.
- `Designate(path, survivor)` wraps `p1@path` into `Strict*(p1@path, survivor)` in place, making `survivor` the *designated survivor* of `p1@path`. The need for the distinguished discipline `Strict*` is explained in §3.4.5.
- `Undesignate(path)` collapses the `Strict*(A, B)` that lives at `p1@path` into `B`, with `B` inheriting the slot and per-arm `meta?`. `A` must be empty.
- `ChangeRoot(path)` promotes `p1@path` to the new root, discarding the ancestor chain above it. The ancestor chain must be a unary "vine". The `path` argument is interpreted in the pre-edit tree; in the post-edit tree the same node sits at `[]`.

When `p1` $=_R$ `p2`, the planner emits the empty sequence (§4), and the live control is left untouched.

3.4 Per-Production Soundness

The two demands that we stated at a high level in §3.3 turn into five concrete obligations that we must discharge for every production of the grammar.

- **Hardware-realizable** creates four obligations. For the substrate to install $[\![\delta]\!]$ atomically, the per-production rule must specify what $[\![\delta]\!]$ does, produce a target that is a valid control with fully specified state, and leave the user's `pop` stream undisturbed. Each of these is a concrete obligation:
  - *Definition.* Specify $[\![\delta]\!]$: `control` ⇀ `control` together with the preconditions under which it is defined; outside that region δ is *incompatible* with `C` and $[\![\delta]\!]$(`C`) is undefined.
  - *Preservation of* $\vdash$. $\vdash$ `C` implies $\vdash$ `C'`. Any packets or index entries that $[\![\delta]\!]$ drops or adds must leave the per-node pifo and packet counts in balance.
  - *Preservation of state.* At every node structurally shared between `C` and `C'` and outside the production's local edit site, the local `state` is preserved verbatim. At the edit site, and at every node of a freshly-spawned subtree, the state is exactly what the production's `init_node_D` / `init_slot_D` (§3.2) invocation prescribes.
  - *Preservation of observation.* Modeling $[\![\delta]\!]$ as a function pins down what it means to be "atomic": the substrate commits the rewrite specified by $[\![\delta]\!]$ *between two consecutive* `push`/`pop` *operations*, so there is no intermediate state for downstream sections to reason about. We defer to §6 that the substrate can slip this change in. In this section, we must show per production that

the commit is *invisible to the user's* pop *stream*. Concretely: every in-flight packet in C sits in some pifo that survives verbatim into C', and no live pifo entry is rewritten, so a pop immediately after δ fires returns exactly what a pop immediately before would have returned.

- **Pol-explainable** creates one obligation:
  - *Characterization.* Give den(δ) in closed form and prove $\lfloor C' \rfloor =_R \mathsf{den}(\delta)(\lfloor C \rfloor)$. The equation is up to $=_R$, not literal =: den returns a definite representative of an $=_R$-class, and $[\![\delta]\!]$ is free to land on any other representative of the same class. Transaction-only effects (e.g., Quiesce's shrinking of z's domain) sit outside den; they are fixed by the operational rule, not by this equation.

Write C for the pre-edit control and C' for the post-edit control. The last four obligations assume $[\![\delta]\!](C)$ is defined; outside the region the production's Definition carves out, δ is *incompatible* with C and $[\![\delta]\!](C)$ is undefined. We do not repeat this boilerplate at every production.

For transaction-only productions (those whose effect lives entirely in some z, like Quiesce and ChangeMeta), den(δ) is the identity on pol and Characterization reduces to $\lfloor C' \rfloor =_R \lfloor C \rfloor$ by inspection: no node's node_state, slot_states, pifo, or child list moves, and z is not visible at pol-level. We still state den explicitly at each production for uniformity.

Several productions (Add, Quiesce, Remove, Designate) modify z at proper ancestors of the edit site, even though those nodes' state is preserved verbatim. There is no separate obligation for these z edits: their pol-invisible content is absorbed by Characterization (which is up to $=_R$, and z does not appear in pol), and their effect on the in-flight stream is absorbed by Observation (every live pifo entry survives a z edit verbatim).

The denotational rules below read, overwrite, and shrink child lists. Let us fix some notation. We write ts[i] for the i-th child and ts[t/i] for ts with its i-th child overwritten by t; this leaves the arity unchanged. ts[-/k] drops the k-th child; later children shift left. Indices follow the path convention of §3.3.

Several productions edit a single arm relative to its parent. When the per-production rule hinges on this relation, we destruct δ's path as $\pi +\!\!+ [k]$, with π the parent's path and k the local index by which that parent reaches the target.

As a warm-up, we discharge the five obligations in some detail for the production Add. The remaining productions reuse the same obligations and arguments, so for them we present only what differs in substance: the closed-form den, the operationally interesting bits of the per-node rule, and the points where any of the preservation arguments departs from Add's.

3.4.1 Add(π, arm, meta?). We rename §3.3's positional arguments for clarity: π : path is the path to the parent node under which the new arm goes, and arm : pol is the policy of the new arm; meta? is the discipline-dependent per-arm metadata of §3.3.

Definition. We must define $[\![\mathsf{Add}(\pi, \mathtt{arm}, \mathtt{meta?})]\!] : \mathtt{control} \rightharpoonup \mathtt{control}$ together with the preconditions on the already running control under which Add is defined.

Say the presently running control is C. Intuitively, Add inserts a freshly compiled subtree ⌈arm⌉ as the new last child of the subtree C@π. It also extends the scheduling transaction z at each proper ancestor of ⌈arm⌉, so that traffic can reach ⌈arm⌉.

$[\![\mathsf{Add}(\pi, \mathtt{arm}, \mathtt{meta?})]\!](C)$ is defined when:

- the path π resolves in C to an internal node,
- meta? matches the slot-initialization schema of the discipline at that node (present iff init_slot_D for that discipline requires it), and
- arm's leaf labels are disjoint from those of C.

The disjointness precondition rules out *splitting* an already-running flow into two (e.g., splitting gmail into gmail_business and gmail_personal): such a split has to relocate already-admitted packets, which Add cannot do.

The transition $C' = [\![\mathsf{Add}(\pi, \mathtt{arm}, \mathtt{meta?})]\!](C)$ is stated per node. Let $k = |C@\pi.\mathtt{slot\_states}|$ be the new arm's slot index.

- *At each proper ancestor of π:*
  - node_state, slot_states, and pifo are preserved verbatim.
  - The local z is extended to admit packets that classify into the new subtree, mapping them to whichever child slot lies on the path down to π. The new entries do not collide with existing ones: the disjointness precondition forces the new leaf labels to lie outside every ancestor z's domain in C, so the extension only adds to z's domain.
- *At C@π:*
  - node_state is unchanged.
  - C'@π.slot_states equals

C@π.slot_states$+\!\!+$[init_slot_D(C@π.node_state, meta?)].

In English: we call init_slot_D to find the per-arm bookkeeping that is needed for a new arm under D, and we append that bookkeeping in at the end. Our arm-order freedom (§3.2) lets us simply append the new child.

  - pifo is unchanged: no pre-existing entry needs renumbering, and the new arm holds no packets yet.
  - C'@π.z extends C@π.z to admit packets that classify into the new subtree, mapping them to child index k at C@π.
- *At every node inside* ⌈arm⌉*:* the local control is ⌈arm⌉'s, verbatim.
- *At every other node (outside* ⌈arm⌉ *and not* C@π *or one of its proper ancestors):* the local control is preserved verbatim.

Preservation of $\vdash$. We must show that $\vdash C$ implies $\vdash C'$: any pifo entries or packets that $[\![\mathsf{Add}(\pi, \mathtt{arm}, \mathtt{meta?})]\!]$ introduces must leave the per-node pifo and packet counts in balance.

At C@π, C'@π.pifo = C@π.pifo contains no entry equal to k (no pre-existing entry can name a slot that did not exist in C), and the new arm at slot k holds no packets, so its well-formedness count reads 0 = 0. Every other slot at C@π keeps its index, its packets, and its entries in C'@π.pifo verbatim, so its matched count is inherited. Every other node's pifo is untouched (the ancestor z extensions touch no pifo at this instant; they only affect the classification of packets that arrive later). Nothing needs repair.

Preservation of state. We must show that at every node structurally shared between C and C' and outside the edit site, the local state is preserved verbatim; and that at the edit site, and at every node of the freshly spawned subtree, the state is exactly what init_node_D / init_slot_D (§3.2) prescribes.

Outside the edit site the local control (and thus its state) is preserved verbatim, including at each proper ancestor of π, where only z changes. Inside $\lceil$arm$\rceil$, every node's node_state and slot_states are what init_node_D / init_slot_D (§3.2) prescribe, by construction of $\lceil$arm$\rceil$. At C@π, node_state is unchanged and slot_states is appended with exactly init_slot_D(C@π.node_state, meta?), as required at the edit site.

Preservation of observation. We must show that the commit is invisible to the user's pop stream: every in-flight packet in C sits in some pifo that survives verbatim into C', and no live pifo entry is rewritten, so a pop immediately after δ fires returns exactly what a pop immediately before would have returned.

No in-flight packet straddles δ's firing. At the instant δ fires, every packet sits in some pre-existing node's pifo, and each such packet survives into the same pifo at the same slot index in C'. The new slot k holds nothing, and no pifo entry is rewritten. So a pop immediately after δ fires returns exactly what a pop immediately before would have. Observation is preserved.

Characterization. We must state den(Add(π, arm, meta?)) in closed form and prove $\lfloor C' \rfloor =_R$ den(Add(π, arm, meta?))($\lfloor C \rfloor$).

We define den by recursion on π:

```
den(Add([], arm, meta?)) (D ts) =
    D ( ts ++ [arm] )
den(Add(i :: rest, arm, meta?)) (D ts) =
    D ( ts[ den(Add(rest, arm, meta?))
                (ts[i]) / i ] )
```

The base case applies once $\pi = []$: the recursion has reached the node where the new arm goes, and arm is appended as the last child. The recursive case walks one step deeper into child i and writes the result back in place. The meta? argument is threaded through the recursion but is not consumed here: it is per-arm bookkeeping consumed by init_slot_D at the operational level.

Proof of characterization. We argue that the structural skeleton of C' matches den(Add(π, arm, meta?))($\lfloor C \rfloor$). The operational rule above leaves every pre-existing arm structurally intact (the z extensions along the ancestor chain are pol-invisible) and adds a new arm at C@π. That new arm is exactly $\lceil$arm$\rceil$. Descending down the path π in $\lfloor C \rfloor$ traces the recursion step for step: at each proper ancestor, step into the child named by π; at the node π reaches, append arm to the child list. For Add, $\lfloor C' \rfloor$ and den(Add(π, arm, meta?))($\lfloor C \rfloor$) are in fact *equal* as pol-trees, not just $=_R$-equivalent: at each proper ancestor of π the child lists agree pointwise, and at C@π both child lists are ts ++ [arm]. We still phrase the characterization mod $=_R$ to keep a uniform shape across productions: $\lfloor C' \rfloor =_R$ den(Add(π, arm, meta?))($\lfloor C \rfloor$).

3.4.2 *ChangeMeta*(τ, meta). τ : path is the path to the target arm whose per-arm metadata changes; it is non-empty (see precondition below) and we destruct it as $\pi +\!\!+ [k]$, with π the path to the parent and k the slot at that parent. The payload meta is interpreted per the parent's discipline: a priority rank when the parent runs Strict, a weight when it runs WFQ.

Say the presently running control is C. Intuitively, ChangeMeta overwrites C@π.slot_states[k]'s per-arm metadata with the new value provided.

Definition. $[\![$ChangeMeta(τ, meta)$]\!]$(C) is defined when τ is non-empty and the parent at π runs Strict or WFQ.

- *At C@π:* node_state is unchanged; the per-arm metadata field of slot_states[k] (the priority rank under Strict, the weight under WFQ) is overwritten with the new value. Every other field of slot_states[k] is preserved verbatim. Under WFQ this includes the virtual-finish tag that records how far slot k has run in the current round. Other slot_states, pifo, and z are unchanged.
- *Everywhere else:* preserved verbatim.

Preservation. Preservation of $\vdash$: no pifo entry is rewritten, no subtree is touched, no packet is relocated, so well-formedness is inherited everywhere. Preservation of state: the targeted metadata field is the intended edit; every other field at C@π and everywhere else is verbatim. Preservation of observation: ranks for in-flight pifo entries were determined when they were pushed. They remain fixed; the new metadata affects only the rank computation for future pushes. So a pop immediately after this $[\![$ChangeMeta(τ, meta)$]\!]$ returns exactly what a pop immediately before would have.

Note: Why the virtual-finish tag is preserved (WFQ). When the parent runs WFQ, the virtual-finish tag at slot k is deliberately *preserved* and not *reset*: resetting would either reward or penalize the targeted arm by yanking it out of the current round, whereas init_slot_WFQ (§3.2) was designed precisely to let a fresh arm "join the current round" without disturbing siblings, and the same principle applies to a weight change on an already-running arm. Under Strict the slot carries no comparable round-bookkeeping field, so the question does not arise.

Characterization.

```
den(ChangeMeta(t, m)) p = p
```

is defined whenever τ resolves in p and the parent at π runs Strict or WFQ. Every node's node_state, child list, pifo, and slot_states.discipline are preserved verbatim: the rewritten field is in slot_state, which the compilation rule strips. So $\lfloor C' \rfloor = \lfloor C \rfloor$. Hence $\lfloor C' \rfloor =_R$ den(ChangeMeta(τ, meta))($\lfloor C \rfloor$).

3.4.3 *Quiesce*(τ). τ : path is the path to the target subtree to silence.

Say the presently running control is C. Intuitively, Quiesce restricts zs at various parts of the control to ensure that any packet bound for a leaf under C@τ is refused admission.

Definition. The topology, the discipline at every node, and every arm's slot index are unchanged. $[\![$Quiesce(τ)$]\!]$(C) is defined whenever τ resolves to a node in C. The empty path is permitted: a root-Quiesce silences every leaf in the tree. There is no precondition on the subtree's contents: Quiesce is the production that stops further admissions, regardless of what is currently held below C@τ.

Let T denote the set of leaves of $\texttt{C@}\tau$ (the leaves we are silencing), and let A denote the union of their ancestor chains: every internal node within $\texttt{C@}\tau$'s subtree, the node $\texttt{C@}\tau$ itself, and every ancestor of $\texttt{C@}\tau$ up to and including the root.

- *Outside $T \cup A$:* the local control is preserved verbatim.
- *At each leaf L in T:* `node_state`, `slot_states`, and `pifo` are preserved verbatim (including any packets L currently holds). $\texttt{C}'\texttt{@}L.\texttt{z}$ restricts $\texttt{C@}L.\texttt{z}$: every packet that L used to admit is removed from $\texttt{C}'\texttt{@}L.\texttt{z}$'s domain.
- *At each internal node n in A:* `node_state`, `slot_states`, and `pifo` are preserved verbatim (including any pre-existing entries pointing toward the quiesced subtree, which continue to be honored on pop). $\texttt{C}'\texttt{@}n.\texttt{z}$ restricts $\texttt{C@}n.\texttt{z}$: packets whose classifier predicate matches any leaf label in T are no longer in $\texttt{C}'\texttt{@}n.\texttt{z}$'s domain. For surviving inputs the output is $\texttt{C@}n.\texttt{z}$'s output verbatim.

Preservation. Preservation of $\vdash$ and state: every node's `node_state`, `slot_states`, `pifo`, and child subtree are preserved verbatim, so well-formedness counts are inherited everywhere and `state` is preserved at every node. The `z` restrictions across $T \cup A$ touch no `pifo` entry and no stored packet. They only refuse future pushes. Preservation of observation follows from §3.4.1's argument: every in-flight packet sits in some pre-existing `pifo` that survives verbatim, so a pop immediately after this $[\![\text{Quiesce}(\tau)]\!]$ returns exactly what a pop immediately before would have.

Characterization. `Quiesce` is `pol`-invisible: its entire effect lies in `z`'s domain, which `pol` does not record.

```
den(Quiesce(t)) p = p
```

is defined whenever τ resolves in `p`. Every node's `node_state`, `slot_states`, `pifo`, and child list are preserved verbatim. Only the `z`s at nodes in $T \cup A$ are restricted, and `z` is not visible at `pol`-level. So $\lfloor \texttt{C}' \rfloor =_R \lfloor \texttt{C} \rfloor = \texttt{den}(\text{Quiesce}(\tau))(\lfloor \texttt{C} \rfloor)$.

Note: Why touch every node on every quiesced push path. Under §3.2's parallel push, every node on the path from a packet's destination leaf to the system root mints an index via its local `z` independently of the others. If we restricted `z` only at a strict subset of those nodes (only the root of the tree, or only inside $\texttt{C@}\tau$), the rest would happily mint indices and enqueue them, leaving the tree in a malformed state. `Quiesce` therefore restricts uniformly, at the leaf (so it refuses to enqueue) and at every ancestor of every quiesced leaf (so no one mints a stray index). The cost is touching $|T|$ leaves and the union of their ancestor chains; the gain is a rejection that preserves $\vdash \texttt{C}'$.

3.4.4 Remove(τ). As before, τ : `path` is the non-empty path to the target arm being removed; we destruct it as $\pi +\!\!+ [k]$ with π the path to the parent and `k` the slot at that parent.

Say the presently running control is `C`. Intuitively, `Remove` unhooks the subtree at slot `k` of $\texttt{C@}\pi$ and renumbers higher siblings down by one. It is defined only when the subtree at τ is empty; we explain this restriction below.

Definition. $[\![\text{Remove}(\tau)]\!](\texttt{C})$ is defined when τ is non-empty and resolves to a node in `C`, the subtree at τ is empty, and $\texttt{C@}\tau$ does not carry the Strict* flag (the planner reaches a Strict* only through `Undesignate`, see §3.4.6).

The topology loses the arm at slot `k` of $\texttt{C@}\pi$; arms at slots $0, \ldots, k-1$ keep their indices; arms at slots $k+1, \ldots$ shift down by one.

- *Outside $\texttt{C@}\pi$ and the removed subtree:* the local control is preserved verbatim. This includes every proper ancestor of π (whose `z` is unchanged) and every sibling arm at $\texttt{C@}\pi$ other than slot `k`, together with the subtrees under them.
- *At $\texttt{C@}\pi$:*
  - `node_state` is unchanged.
  - `slot_states` $= \texttt{C@}\pi.\texttt{slot\_states}[-/k]$: the entry at slot `k` is dropped. Entries at slots $> k$ shift down by one to track the renumbered arms.
  - `pifo`: the precondition (subtree at τ is empty) plus $\vdash$ `C` forces $\texttt{C@}\pi.\texttt{pifo}$ to contain *no entry equal to* `k`, so no entry is deleted; whatever bookkeeping is needed to keep the surviving entries pointed at their (now-resident-at-arity-$n-1$) children must be done. If one were literally indexing children by their positions, that would mean decrementing entries $> k$ by one. Implementations have ways around even this; for instance, the substrate model of §6.1 keys children by stable per-edge handles, so survivors' pifo entries are unchanged.
  - `z` is restricted on inputs and renumbered on outputs. Packets that $\texttt{C@}\pi.\texttt{z}$ would have routed to slot `k` are no longer in $\texttt{C}'\texttt{@}\pi.\texttt{z}$'s domain. For surviving inputs, an output of `(i, r)` with $i > k$ becomes `(i - 1, r)`. Slots $< k$ are untouched on either axis.

A standalone `Remove` thus restricts classification at $\texttt{C@}\pi$ so that packets bound for the removed subtree are dropped at $\texttt{C@}\pi$ rather than being routed to a non-existent slot. In the typical planner usage, a preceding `Quiesce` has already restricted every `z` on every push path to the soon-to-be-removed leaves (§3.4.3), so no new packet bound for the removed subtree is even being admitted; the input restriction at $\texttt{C@}\pi$ is then redundant but harmless, while the output renumbering at $\texttt{C@}\pi$ is still needed.

Preservation. Preservation of $\vdash$: at $\texttt{C@}\pi$, the precondition gives $\texttt{C@}\pi.\texttt{pifo}$ no entry equal to `k`, so no pifo entry is deleted; surviving slots $i' < k$ of $\texttt{C}'\texttt{@}\pi$ inherit their matched counts from slot i' of $\texttt{C@}\pi$, and surviving slots $i' \geq k$ inherit theirs from slot $i'+1$ (same subtree). Every proper ancestor and every surviving sibling is verbatim, so its well-formedness count is inherited. Preservation of state: every proper ancestor and surviving sibling preserves its `state` verbatim. No `init`-rule fires, and `slot_states` drops slot `k`'s entry while every other entry is preserved verbatim, with its position shifted to match the new arm order. Preservation of observation follows from §3.4.1's argument applied to the surviving structure: every in-flight packet sits in a preserved `pifo` at the same slot (if $< k$) or one slot lower (if $> k$), and every surviving `pifo` entry points to the same child subtree after renumbering, so a pop immediately after this $[\![\text{Remove}(\tau)]\!]$ returns exactly what a pop immediately before would have. The restriction at $\texttt{C}'\texttt{@}\pi.\texttt{z}$ comes into play only for pushes that arrive after this $[\![\text{Remove}(\tau)]\!]$ fires.

Characterization.

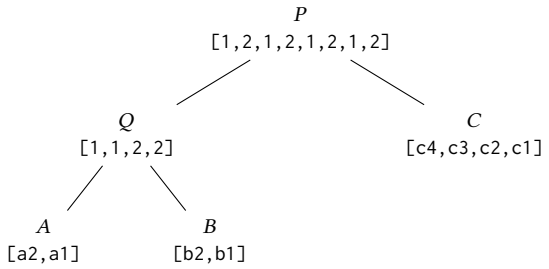
```
den(Remove([k])) (D ts)
    = D ( ts[-/k] )
den(Remove(i :: rest)) (D ts)
    = D ( ts[ den(Remove(rest)) (ts[i]) / i ] )
    when rest is non-empty
```

The base case fires once τ has been walked down to the parent of the removed arm: the k-th child is dropped from the arm list. The recursive case walks one step deeper into child `i` and writes the result back in place. $\mathsf{den}(\mathsf{Remove}(\tau))$ is defined when τ is non-empty and resolves in the input pol; the emptiness precondition that $[\![\mathsf{Remove}(\tau)]\!]$ imposes is operational, not visible at `pol` level.

Proof shape is the same as `Add`'s: outside the removed subtree every node is structurally intact (ancestors verbatim, surviving siblings verbatim), and at $\mathtt{C@}\pi$ the child list shrinks by dropping slot k. So $\lfloor \mathtt{C}' \rfloor =_R \mathsf{den}(\mathsf{Remove}(\tau))(\lfloor \mathtt{C} \rfloor)$.

Note: Why an empty subtree. Deleting an *occupied* child has no canonical local realization. Consider `P(Q(A, B), C)`.

Diagram legend. Each internal node is labeled with its index-PIFO and each leaf with the packets it holds. Each PIFO is drawn with its most favorably ranked entry (the next one to be popped) on the far right; so a leaf labeled `[a2,a1]` releases `a1` before `a2`, and an internal-node PIFO `[1,2,1,2]` will yield slot indices `2, 1, 2, 1` in pop order. A struck entry like ~~`1`~~ is one we have dropped from the PIFO.

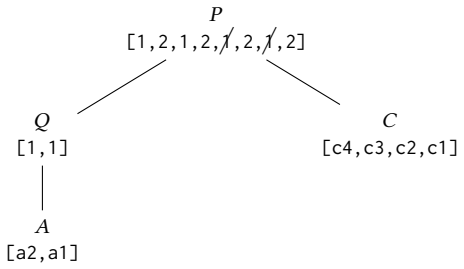


Here `Q` refers to `A` using the index 1, and to `B` using 2. `P` refers to `Q` using 1 and to `C` using 2. The tree is well-formed. `P` appears to be running a round-robin style policy and `Q` appears to be prioritizing `B` strictly over `A`. Say the user asks us to delete `B`, dropping its packets. To restore well-formedness, we need to remove two instances of the index 2 from `Q`'s PIFO and two instances of the index 1 from `P`'s PIFO.

At `Q` the edit is unambiguous. Well-formedness of the original tree forces `Q`'s PIFO to have exactly two instances of 2, and we delete those two. The trouble is one level up. `P`'s PIFO originally had four instances of 1, and well-formedness demands that we remove two of them. But *which* two? No entry in `P` carries the meta-information "I was enqueued when a packet was inserted into `B`"; an entry 1 in `P` only means "when this index is popped, recursively ask subtree `Q` to emit the next packet" (see §2.1). This absence of meta-information is a feature of PIFO trees, not a defect: an entry is deliberately detached from the packet it was enqueued with, which is what lets each node schedule its own discipline in isolation and lets a subtree be reconfigured without rewriting its ancestors. We could make deletion unambiguous by tagging each entry at push with the leaf it is destined for, but that would discard the abstraction: every internal node would then have to track the entire downstream structure, and the composability that makes PIFO trees scale would be lost.

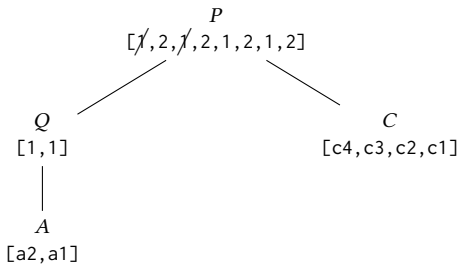
Given this, *we can only pick two instances of* `1` *arbitrarily.* Our choice is, in effect, a scheduling decision, as it affects how `P` intermixes `C` and `Q` traffic. For example:

a. *Keep the two leftmost* `1`*s* (strike the two on the right):



Six pops drain `P` and yield `c1, c2, c3, a1, c4, a2`: `A` is pushed to the back, both its packets emerging only after `C`'s run is mostly spent.

b. *Keep the two rightmost* `1`*s instead* (strike the two on the left):



The same six pops now yield `c1, a1, c2, a2, c3, c4`: this is exactly the round-robin schedule `P` was configured for, alternating `C` and `A` until `A` is exhausted and `C` drains alone.

Same tree, same surviving packets, two different service orders. The *choice of which* `1`s to drop *silently reschedules* `A` *against* `C`, and nothing in `p1` or the deletion request records which the operator wanted.

We do not wish to make a scheduling decision for the operator, so our only other option is to reconstruct the tree as if `B` had never been admitted: drain the tree completely, filter the `B`-packets, and re-push, recomputing every rank and cursor along the way. This is a stop-the-world rebuild.

Our grammar sidesteps the question entirely. By draining `B` to empty before removing it (§4), every pop that served a packet from `B` has already removed the matching index-entries from `B`'s ancestors in the ordinary course of `pop`: a packet leaving `B` consumes a 2 at `Q` and a 1 at `P`, bringing each ancestor's count into accord with its post-drain contents. At the instant `Remove` fires, `Q`'s 2-count is zero (`B` is empty) and `P`'s 1-count is exactly the number of packets still under `Q` (here 2, all attributable to `A`). There is nothing left to reconcile and no hidden policy choice to make: the structural deletion is unambiguous.

3.4.5 Designate(τ, `survivor`). τ : `path` is the path to the target subtree being wrapped, and `survivor : pol` is the designated-survivor policy that becomes the wrap's low-priority arm.

Say the presently running control is C. Intuitively, Designate replaces C@τ in place with Strict*(C@τ, ⌈survivor⌉).

Strict* is the same discipline as Strict, distinguished only by a one-bit designated flag that each node carries: Designate sets the bit, Undesignate (§3.4.6) clears it. Semantically the two are identical; every push, pop, and well-formedness check treats Strict*(A, B) exactly as Strict(A, B), so the obligations below are stated against the ordinary Strict semantics of §3.2. The star exists only so that Undesignate's precondition ("τ lands on a Strict*") and a hardware-level argument in §6 (that Strict* adds no PE depth) can be stated structurally. The §3.2 DSL in which the operator writes policies can parse only Strict, never Strict*, so a Strict* is unreachable in any user-written pol and arises only in the middle of a planner sequence, between a Designate and its eventual Undesignate. The wrap is therefore pol-visible only as a Strict.

Definition. ⟦Designate(τ, survivor)⟧(C) is defined when τ resolves to a node in C and survivor's leaf labels are disjoint from those of C outside the subtree C@τ. Overlap with C@τ's own leaves is explicitly permitted; we explain why in a separate Note below. Let N denote the freshly introduced Strict* node, and let P_0 denote the number of packets currently held under C@τ.

- *Outside the subtree at τ, outside N, and off the ancestor chain to τ:* preserved verbatim. If $\tau = \pi +\!\!+ [k]$ is non-empty: at C@π the node_state, slot_states, and pifo are unchanged; the slot k that used to point to C@τ now points to N, inheriting C@π.slot_states[k] (the per-arm meta is held by C@π, not by N, so the wrap is transparent to the parent); z is extended as described in the next bullet.
- *At each proper ancestor of τ (including the root, and including π):* node_state, slot_states, and pifo preserved verbatim. The local z is extended to admit packets that classify into the new arm-1 subtree under N, mapping them to whichever child slot at this ancestor lies on the path down to N. Pre-existing mappings are untouched. This is the exact analog of Add's ancestor z-extensions (§3.4.1).
- *At N:* for any priority ranks hi < lo, set node_state = init_node_Strict() and

```
slot_states = [init_slot_Strict(node_state, hi),
               init_slot_Strict(node_state, lo)].
```

  Arm 0 is the old C@τ (verbatim, including its entire subtree and contents); arm 1 is ⌈survivor⌉; pifo is seeded with P_0 entries with slot value 0 and rank hi (one per packet still held under arm 0). z routes packets by leaf label: a packet whose label exists in ⌈survivor⌉ goes to slot 1, otherwise (label exists only in the old subtree) it goes to slot 0. This rule is total on N's admitted labels and resolves the overlap case toward the survivor: a label shared between the old subtree and survivor belongs to the survivor from δ's firing onward, while the residual packets under arm 0 drain on their existing entries.
- *Inside* ⌈survivor⌉*:* the local control is ⌈survivor⌉'s, verbatim.

If τ is empty, the entire control becomes N, with the old root sitting as arm 0.

Note: Overlap between the old subtree and survivor*.* Overlap of leaf labels between C@τ and survivor is explicitly permitted. The moment δ fires becomes the boundary between old traffic (which continues to drain from arm 0, C@τ) and new traffic (which arm 1 admits): from this instant on, an arriving packet whose label appears in ⌈survivor⌉ is routed by N.z to slot 1, while the residual packets already under arm 0 drain on their existing pifo entries. The two arms thus stay operationally disjoint even when their label sets overlap, which is the very property that lets Designate express in-place subtree replacement (§4).

Preservation. Preservation of ⊢ holds at N: by construction the 0-count in N.pifo equals the packet count under arm 0, and arm 1 holds no packets and no 1-entries in N.pifo. Every node inside arm 0 is preserved verbatim, so its well-formedness count is inherited. At C@π (if τ non-empty), pifo is unchanged, and the slot-k count $P_0 + 0$ matches the count that C@τ had under C@π. The ancestor z-extensions touch no pifo and route no in-flight packet, so they affect only pushes that arrive after this ⟦Designate(τ, survivor)⟧. Preservation of state is the standard split: arm 0 is verbatim; arm 1 is ⌈survivor⌉ by construction; N's node_state and slot_states come from the listed init calls; everything else is verbatim. Preservation of observation: every in-flight packet under arm 0 sits at exactly the same position it did in C@τ, since arm 0 is C@τ verbatim. If $P_0 > 0$, the smallest entry in N.pifo is 0, so any pop that descends to N continues into arm 0 and returns exactly the packet a pop before δ fired would have. If $P_0 = 0$, then by ⊢ C no pop ever descends to N (the subtree at τ was already empty), so the question is vacuous. Arm 1 first sees traffic only after a future push.

Characterization.

```
den(Designate([], survivor)) p =
    Strict(p, survivor)
den(Designate(i :: r, survivor)) (D ts) =
    D ( ts[ den(Designate(r, survivor))
                (ts[i]) / i ] )
```

den returns Strict, not Strict*: the designated bit is pol-invisible. The proof is the same shape as Add's: the operational rule leaves every node outside the wrap structurally intact, the new arm 1 is ⌈survivor⌉, and the new Strict* reads as Strict at pol-level. So $\lfloor C' \rfloor =_R$ den(Designate(τ, survivor))($\lfloor C \rfloor$).

3.4.6 Undesignate(τ). τ : path is the path to the target Strict* node being unwrapped.

Say the presently running control is C. Intuitively, Undesignate unwraps the Strict*(A, B) at C@τ whose A-arm is empty, leaving B in its slot. It is the structural inverse of Designate.

Definition. ⟦Undesignate(τ)⟧(C) is defined when τ resolves to C@τ, C@τ carries the designated bit (so it is a Strict* introduced by an earlier Designate), and arm 0 of C@τ is empty. Let N = C@τ.

- *Inside B (arm 1 of N):* preserved verbatim, every node.
- *At* C@π (if $\tau = \pi +\!\!+ [k]$ is non-empty): node_state, pifo, and z unchanged; slot_states[k] preserved verbatim (the wrapper inherited it at Designate time, and the unwrap returns it unchanged); slot k now points to B rather than to N. N itself and its arm-0 stub are discarded.

- If τ is empty, the new root is B, inheriting B's own `node_state`, `slot_states`, `pifo`, `z`, and child list verbatim; N and its arm-0 stub are discarded.

Preservation. Preservation of $\vdash$: at $\texttt{C@}\pi$, `pifo` is unchanged; slot `k`'s count was $0 + |B|$ under N and is now $|B|$ under B directly, so the count matches. Inside B, every node is verbatim. The discarded N.`pifo` was carrying its own count of arm-1 entries one level too deep: by $\vdash$ `C` and the empty-arm-0 precondition, that count equals $|B|$, which is exactly what $\texttt{C@}\pi\texttt{.pifo}$'s `k`-entries already carry one level up. After the unwrap, those `k`-entries route directly to B, whose pifo count is still $|B|$, so the well-formedness obligation at $\texttt{C@}\pi$ reads $|B| = |B|$ and inside B reads as before. The previously-redundant routing step ("Strict* says go to arm 1," which arm 0 being empty had already forced) is the only step elided. Preservation of state: standard verbatim everywhere outside N; N's local state vanishes. Preservation of observation: arm 0 holds no packet by precondition, so Strict*'s "favor arm 0" behavior was already routing every pop to arm 1, and the unwrap preserves this routing exactly.

Characterization.

```
den(Undesignate([])) (Strict(A, B))
    = B
den(Undesignate(i :: r)) (D ts)
    = D ( ts[ den(Undesignate(r)) (ts[i]) / i ] )
```

The base case is defined when the input matches Strict(_, _); the operational rule additionally requires that arm 0 of $\texttt{C@}\tau$ be empty at the moment δ fires. The base case consumes the Strict wrapper (which on the `pol` side is what the Strict* reads as). Proof shape is `Remove`'s: outside N nothing structural moves, and at N the Strict wrapper is dropped. So $\lfloor \texttt{C}' \rfloor =_R \texttt{den}(\texttt{Undesignate}(\tau))(\lfloor \texttt{C} \rfloor)$.

3.4.7 ChangeRoot(v). v : `path` is the path to the subtree being promoted to the new root.

Say the presently running control is `C`. Intuitively, `ChangeRoot` promotes $\texttt{C@}v$ to the new root, discarding every ancestor above it. It is the only production that erases structural ancestors, and the only one whose `pol`-level effect is to shrink the tree from above rather than edit it locally. The mechanics are short; the notes below are substantive.

Definition. $[\![\texttt{ChangeRoot}(v)]\!](\texttt{C})$ is defined when v is non-empty, resolves to a node in `C`, and every internal node strictly above $\texttt{C@}v$ is *unary*, with its sole arm continuing toward $\texttt{C@}v$. Under these preconditions the discarded ancestor chain carries scheduling metadata but no off-path traffic.

- *At $\texttt{C@}v$ and inside its subtree:* preserved verbatim. The result $\texttt{C}'$ is exactly $\texttt{C@}v$ standing alone as a tree.
- *At every proper ancestor of v:* discarded entirely, together with the local `node_state`, `slot_states`, `pifo`, and `z`.

Preservation. Preservation of $\vdash$: $\vdash$ `C` is a conjunction of per-node well-formedness obligations, and the subset of those obligations attached to $\texttt{C@}v$'s nodes constitutes $\vdash \texttt{C@}v$, which is exactly $\vdash \texttt{C}'$. The discarded ancestors' obligations are simply forgotten. Preservation of state: standard verbatim within $\texttt{C@}v$; ancestor state is gone. Preservation of observation: every in-flight packet sits in some `pifo` within $\texttt{C@}v$'s subtree (the ancestor `pifo`s held routing entries, not packets), all preserved verbatim. A pop immediately after this $[\![\texttt{ChangeRoot}(v)]\!]$ returns exactly what a pop immediately before would have: a pop before δ fires descends through the unary chain by popping each ancestor's PIFO in turn, and since each such PIFO has a single slot (slot `0`), the popped entry is forced to be a slot-`0` entry regardless of its rank; the descent therefore lands at $\texttt{C@}v$'s root and pops from there. A pop after δ fires acts on $\texttt{C@}v$'s root directly, returning the same packet.

Characterization.

```
den(ChangeRoot(v)) p = p@v
```

is defined whenever v is non-empty and resolves in `p`. The realizes-relation propagates structurally: if `C` realizes `p`, then $\texttt{C@}v$ realizes $\texttt{p@}v$, since compilation is per-node and the discipline at each surviving node is unchanged. So $\lfloor \texttt{C}' \rfloor =_R \texttt{p@}v = \texttt{den}(\texttt{ChangeRoot}(v))(\lfloor \texttt{C} \rfloor)$.

Notes. Why the unary precondition. The precondition rules out, by construction, any silently-dropped packet-bearing siblings. A non-unary ancestor would have arms branching off the path to $\texttt{C@}v$; those subtrees would vanish with their ancestor, taking their packets with them. Allowing this would make `ChangeRoot` a structural deletion of arbitrarily many off-path subtrees masquerading as a root change, with no semantic story for those packets. The richer reconfiguration of pruning to a subtree whose ancestor chain branches off into packet-bearing relatives is realized as the `PruneDownTo` idiom (§4.1), which first drains and `Removes` those siblings in sequence, reducing the chain above $\texttt{C@}v$ to the unary shape that `ChangeRoot` then accepts.

What the chain carries, and what is discarded. An internal node's `node_state`, rank function, and `pifo` ordering exist to pick among siblings. At a unary node the sibling-picking role degenerates: with one arm, every pop is forced into it.

What can remain active at a unary node is rank computation that depends on per-packet attributes rather than on sibling identity. If a chain's disciplines do no such per-packet reordering, each unary node along the chain is operationally a passthrough, and the immediate-pop observation-preservation above extends to every subsequent pop and push. If they do (LSTF [3] is a canonical example, with rank determined by each packet's deadline), the chain is actively shaping traffic, and `ChangeRoot` removes that shaping. That is precisely the production's intended role: the atomic step by which the operator says "promote $\texttt{p1@}v$ to the root and discard the ancestor shaping above it." The `PruneDownTo` idiom packages this with the upstream draining and `Removes` that make the chain unary in the first place, so the operator's request ("prune to this subtree") is realized as a sequence whose final step discards exactly the ancestor influence that the operator has chosen to abandon.

Pol-level effect. The `pol` changes from the unary chain wrapping $\texttt{p1@}v$ (e.g., $\texttt{Strict}(\texttt{p1@}v)$ or $\texttt{LSTF}(\texttt{p1@}v)$) to just $\texttt{p1@}v$. This is `pol`-visible: the root discipline observably changes, which is the substantive content of the production. Whether that pol-visible change reorders in-flight traffic depends on whether the discarded chain was doing per-packet reordering (see previous Note).

3.5 Preserving this proof down to hardware

§3.4 proves soundness at the δ level, where each production (`Add`, `Quiesce`, `Remove`, ...) carries a well-formed tree to a well-formed tree. To run on hardware, each edit is lowered, by a simple and mechanical compilation, into a sequence of fine-grained instructions in our IR. The IR alphabet and the substrate machinery are the subject of §6; here we only need that the IR has fine-grained verbs (we will name `Isa_spawn`, `Isa_adopt`, `Isa_emancipate`, `Isa_assoc`, `Isa_deassoc`, and the like) and that a single such verb, unlike a whole production of δ, *can* leave the tree malformed: a node freshly created by `Isa_spawn` has not yet been wired to its parent by `Isa_adopt`, for example.

We do not prove soundness at the IR level, but instead informally make the case for why the §3.4 proof survives the lowering. There are two reasons.

- The compilation is *faithful*: each production of δ expands to a fixed instruction sequence that, when run to completion, realizes exactly that production's effect. We give the command-to-commands translation and take its faithfulness to be uncontroversial.
- Our substrate runs each such sequence as a single *transactional commit*: a bracketed group of IR instructions that installs as one instant from the user's perspective, with no `push` or `pop` interleaved between the bracket's open and close (§6.1 spells out the substrate machinery that makes this so). The transiently-malformed intermediate trees are therefore never observed. That commit is precisely how the substrate *realizes* the atomicity property of §3: atomicity asked for an instantaneous control replacement between two user operations, and the commit is what collapses a multi-instruction lowering into one such instant. Every `push`/`pop` therefore still lands on a well-formed control (`p1`, some `link`, or `p2`), exactly as §3.4 proved; the IR's transient malformedness lives entirely inside commits, invisible to the user.

The same argument carries from the IR down to hardware: the hardware executes a committed sequence atomically with respect to user operations, so what it exhibits is again what §3.4 proved. The compilation itself, and the substrate machinery that makes a commit atomic, are the subject of §6.

[AM note for Zhiyuan: the §3.5 argument leans on the substrate supporting *atomic transactional commits*: a multi-instruction lowering must install as a single instant from the user's perspective, so that the transiently-malformed intermediate trees inside a commit are never observed. Our own substrate provides this. The open question for §6 is whether composition with a third-party substrate (e.g., vPIFO) requires the same property and, if so, whether vPIFO offers it. Flagged here so the §3.5 claim "the proof survives the lowering" is not read as substrate-independent.]

4 REALIZING RECONFIGURATIONS AS GUARDED SEQUENCES

This section composes the productions of δ (§3.3) into *guarded sequences* $(\varphi;\delta)^*$, where a guard φ is a predicate on the state of the live control. Guarded sequences realize changes to the live control that no single δ can express.

A guarded sequence threads the live control through a chain of intermediate controls, one per pair of consecutive productions. We call each such intermediate control a `link`, writing $\texttt{link}_i$ for the control on which the `i`-th δ (zero-indexed) fires; $\texttt{link}_0 = \texttt{C}$ is the starting control, and $\texttt{link}_{i+1} = [\![\delta_i]\!](\texttt{link}_i)$ is what the `i`-th δ leaves behind (§4.1 makes this precise).

The headline result of the section, proved below, is that each `link` is itself an ordinary §3.1-style control, so the "transitionary period" *needs no new semantics*: this is Obligation 1 of §1, discharged. Moreover, since every production of §3 is *pol-explainable* (each δ has a $\texttt{den}(\delta)$ that tracks its pol-level effect), we can echo to the operator, at every step of the sequence, the `pol` that the live control actually realizes. The chain of `den`s starting from $\lfloor\texttt{C}\rfloor$ gives $\lfloor\texttt{link}_i\rfloor$ for each `i`, so the operator is never in the dark about what policy is running.

4.1 Guarded Sequences

Grammar.

```
phi    ::= true
         | empty(path)

gseq   ::= eps
         | (phi ; d) ; gseq
         | I ; gseq

I      ::= Retire(path)
         | SlowRetire(path)
         | Replace(path, pol)
         | PruneDownTo(path)
```

We write `gseq` for "guarded sequence." A guard φ is either trivially satisfied (`true`) or asserts that the subtree at `path` is empty; it is always paired with exactly one δ drawn from §3. The informal notation $(\varphi;\delta)^*$ used elsewhere in the paper names exactly the `gseq` form after idiom expansion: a finite sequence of (φ,δ) pairs. An idiom `I` appears in a `gseq` *without* a guard because an idiom expands into a `gseq` of its own whose internal guards carry the synchronization (§4.2). For instance, §4.2's `PruneDownTo` expansion reads

```
Retire(pi_a) ; ... ; Retire(pi_z) ;
    (true ; ChangeRoot(path))
```

mixing bare idiom invocations with one explicitly-guarded δ. After idiom expansion (§4.2), every step is a $(\varphi;\delta)$ pair, and that is the form on which the rest of §4 reasons.

We have a rather small vocabulary of guard forms. Those two have sufficed for every sequence we have needed; we take the minimality as a small design win. The framework does not depend on this minimality, and a richer predicate language can be slotted in without disturbing the rest of §4.

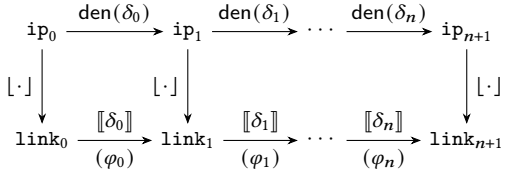
Sequence semantics. The grammar above gives a `gseq` its shape; this subsection gives it operational meaning. §3 wrote `C` for the pre-edit control and `C'` for the post-edit one; here we generalize from a single-step replacement to a chain of steps. A *transition planner* (§5) realizes a reconfiguration from `C` to `C'` as a guarded sequence

```
(phi_0 ; d_0) ; (phi_1 ; d_1) ; ... ; (phi_n ; d_n).
```

Write $\texttt{link}_0 = \texttt{C}$ and $\texttt{link}_{i+1} = [\![\delta_i]\!](\texttt{link}_i)$ for $0 \le i \le n$, with $\texttt{link}_{n+1} = \texttt{C}'$. Pairs fire in order: $\texttt{link}_i$ runs and serves ordinary pushes and pops until φ_i becomes true on its state, at which instant δ_i fires and produces $\texttt{link}_{i+1}$; only then is the next pair $(\varphi_{i+1}; \delta_{i+1})$ in play. Crucially, φ_{i+1} is evaluated on $\texttt{link}_{i+1}$, which exists only after δ_i has fired, so the sequence is genuinely sequential and not a set of independent guards racing on the same control.

A guard may be `true`, in which case δ_i fires the moment $\texttt{link}_i$ is installed, with no waiting; but a `true` later in the sequence is still gated by every preceding pair. For instance, the closing (`true`; ChangeRoot(`path`)) of §4.2's `PruneDownTo` is nominally guarded by `true` but in the global timeline cannot fire until every preceding `Retire` has run to completion. The empty sequence is the case $\lfloor \texttt{C} \rfloor =_R \lfloor \texttt{C}' \rfloor$: the live control is left untouched.

The iterated picture. §3.2 drew the unit cell of Rule 3 against a single production: one operational rewrite $[\![\delta]\!]$, one pol-level effect $\texttt{den}(\delta)$, one $\lfloor\cdot\rfloor$ bridge between them. A guarded sequence stacks copies of that unit cell horizontally:



with $\texttt{link}_0 = \texttt{C}$, $\texttt{link}_{n+1} = \texttt{C}'$, and $\texttt{ip}_i := \lfloor \texttt{link}_i \rfloor$. The bottom edge is the operational chain that the substrate runs: each horizontal arrow $\texttt{link}_i \to_{[\![\delta_i]\!]} \texttt{link}_{i+1}$ fires once its guard φ_i becomes true on $\texttt{link}_i$. The top edge is the pol-level chain that the operator sees: each $\texttt{ip}_i$ is a valid `pol` with a readable scheduling semantics, and each $\texttt{den}(\delta_i)$ carries one $\texttt{ip}_i$ to the next. The vertical $\lfloor\cdot\rfloor$ bridges are the per-step Rule-3 cells, one per δ_i, each one already discharged in §3.4.

The diagram contains a mild abuse of notation: we should be writing $=_R$ and $\sim_R$ in some cases instead of pretending that two routes actually coincide on the nose. However, we prefer this style for now to lighten the notation.

The picture pays two dividends. First, every operator-observable intermediate $\texttt{link}_i$ corresponds to an $\texttt{ip}_i$ that the runtime can echo back, just as §3.2 echoed `p1'` from $\lfloor$`C1`$\rfloor$: there is no transient between-the-pols regime that lacks a pol-level meaning. Second, §4.3 will use the top edge directly: the pol-level check on an imperative-mode sequence is the assertion that $\texttt{ip}_{n+1} =_R$ `p2`, with each $\texttt{den}(\delta_i)$ defined on its $\texttt{ip}_i$.

Safety. **Lemma 4.1 (Safety of guarded sequences).** For any guarded sequence $(\varphi_0; \delta_0); \ldots; (\varphi_n; \delta_n)$ emitted by the planner against a well-formed $\texttt{link}_0 = \texttt{C}$, every control the user observes during execution is well-formed: $\vdash \texttt{link}_i$ holds at each `i`, and $\vdash \texttt{C}'$.

Proof sketch. $\vdash \texttt{link}_0$ by assumption. Each δ_i preserves $\vdash$ by the per-production obligation discharged in §3.4, so $\vdash \texttt{link}_{i+1}$. Each $\texttt{link}_i$ itself preserves $\vdash$ under ordinary push/pop, since §3.1's push carries a well-formed PIFO tree to a well-formed one and pop does likewise on non-empty inputs. So the user-observable trace is a stream of ordinary push/pop operations served in turn by well-formed controls $\texttt{C}, \texttt{link}_1, \ldots, \texttt{link}_n, \texttt{C}'$. Each intervening $\texttt{link}_i$ is therefore an ordinary §3.1 control, and the "semantics of `link`" is nothing more than the §3.1 semantics of the controls we install: each δ's job is to be a sound atomic replacement, and the planner's job is to order them behind sensible guards.

The lemma's reach is what discharges Obligation 1 of §1: safety is entirely local, following from the per-production obligations of §3.4 with no global argument about the sequence.

Liveness. Liveness is not a soundness obligation in this paper. A planner that emits an unsatisfiable guard is misbehaving but not unsafe: the live scheduler remains well-formed.

[AM note: a natural extension would be an operator-facing *withdraw* that aborts a stuck in-flight sequence and rolls the live control back to a safe checkpoint, but we have no such mechanism today and no concrete plans to build one; the queueing story of §4.4 is what we commit to.] A sequence reaches `C'` only if each φ_i eventually becomes true on $\texttt{link}_i$, and Lemma 4.1 says nothing about that. A `Quiesced` subtree may never drain if higher-priority siblings starve it; a guard `empty(path)` on a subtree fed by an adversarial higher-priority neighbor may never fire. We therefore treat liveness as a *nice-to-have*, the planner's and the operator's joint concern. Every sequence we emit is safe; whether and how fast it completes is a function of how well the planner chose its guards and of the traffic the live scheduler meets. For the queueing behavior when follow-up requests arrive mid-flight, see §4.4.

4.2 Idioms: Named Sequences

Idioms are an operator-facing vocabulary for frequently-used guarded sequences. They are the typical building block of imperative-mode sequences (§4.3).

An idiom is a macro over the grammar δ (and, recursively, over other idioms). It expands into a fixed $(\varphi; \delta)^*$ sequence: a list of productions with the guards between them spelling out what the system waits for. Soundness is compositional: each step of the expansion is sound by §3.4, and the sequence inherits the sequence-level reasoning above. An idiom expansion that would hit an undefined production on the current control is rejected: the soundness checks fire on the expanded sequence just as they would on a hand-written one.

We name four starter idioms. New ones can be added later without changing the framework, since an idiom is just a named $(\varphi; \delta)^*$.

- **`Retire(path)`** expands to

  (`true`; Quiesce(`path`)); (`empty`(`path`); Remove(`path`)).

  Quiesces the subtree at `path`, waits for it to drain to empty, then `Removes` it. The operator-facing way to say "tear this subtree down gracefully."
- **`SlowRetire(path)`** = (`empty`(`path`); Remove(`path`)). Waits for the subtree at `path` to drain, then `Removes` it. The operator may use this if the subtree at `path` needs to receive a little more traffic but is generally being phased out.
- **`Replace(path, B)`** =

  ```
  (true; Designate(path, B)) ;
  (true; Quiesce(path ++ [0])) ;
  (empty(path ++ [0]); Undesignate(path))
  ```

  Designates `B` as the survivor of the current `pol@path` (which, after Designate, sits at `path ++ [0]` as the first

arm of the inserted Strict* node), quiesces it, waits for it to drain, then collapses the Strict* onto B. At the pol level the composite effect is wholesale replacement of `pol@path` by B; the operator-facing idiom adds the `Quiesce` in the middle so that the original subtree empties out before the collapse fires. The footprint of `Replace(path, B)` is the subtree at `path`, the wrap node inserted at `path` by `Designate`, and the `z` chain at the ancestors of `path` (extended by `Designate`, restored by `Undesignate`); §5 frames confinement against exactly this set.

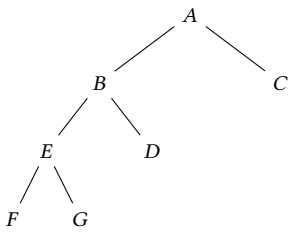
- **`PruneDownTo(path)`** expands to

$$\text{Retire}(\pi_a); \ldots; \text{Retire}(\pi_z); (\texttt{true}; \text{ChangeRoot}(\texttt{path})),$$

where $\pi_a, \ldots, \pi_z$ are paths to the off-path subtrees along the route from the root to `path`. This idiom is the operator-facing way to say "abandon everything except this subtree."

Each `Retire` removes one off-path subtree; once they have all fired, every ancestor along the path is unary, satisfying `ChangeRoot`'s precondition (§3.4.7). (If no off-path subtrees exist, the idiom reduces to `(true; ChangeRoot(path))` alone.) The `Retires` on disjoint off-path subtrees commute, so any ordering suffices; we use root-first as a stable convention.

For instance, suppose the running tree is `A(B(E(F, G), D), C)`:



The index paths to its nodes are:

- A at `[]`
- B at `[0]`
- C at `[1]`
- `E(F, G)` at `[0, 0]`
- F at `[0, 0, 0]`
- G at `[0, 0, 1]`
- D at `[0, 1]`

The operator wants to retain only D, so they issue `PruneDownTo([0, 1])`.

The off-path subtrees are `E(F, G)` (at `[0, 0]`) and C (at `[1]`). So `PruneDownTo([0, 1])` expands to: `Retire([0, 0]) ; Retire([1]) ; (true; ChangeRoot([0, 0]))`. Note that the path `[0, 0]` appears twice in this expansion with different referents: in `Retire([0, 0])` it points to `E(F, G)` (the operand at the moment that `Retire` fires), and in the final `ChangeRoot([0, 0])` it points to D, which moved from `[0, 1]` to `[0, 0]` once `E(F, G)` was retired. A path-resolution system computes these targets automatically at idiom expansion; the operator only writes `PruneDownTo([0, 1])`, naming D by its location at the moment of request.

On the natural-feeling opposite. `PruneDownTo` says "keep this subtree and abandon everything around it"; the natural-feeling opposite would say "keep the running tree and splice some fresh ancestor structure around it" (the running root becoming a subtree of a freshly-introduced context). We do not support this as a production or as an idiom. The obstruction is not topological but stateful: a freshly-spawned ancestor needs a `node_state`, per-arm `slot_states`, a populated index `pifo`, and a `z` whose history matches the live traffic descending into the subtree it now wraps, and there is no honest way to synthesize the state these ancestors would have carried *had they been there all along*. Disciplines that look at packet features or history (WFQ's virtual time, RR's cursor, LSTF's per-deadline ranks) make this concrete: an ancestor introduced cold would mis-schedule the in-flight traffic relative to what an ancestor present from the start would have done. The operator can still reach such a target through a `Replace` at a sufficient ancestor (§5.1's always-available fallback), which freshly compiles the new wrapping and drains the old running subtree through it; what the grammar lacks is a confined way to install the wrap without that drain.

4.3 Authoring modes

Two authoring modes produce sequences, and the operator chooses freely between them.

- In *declarative mode*, the operator writes a `pol`, say `p1`, and, to reconfigure, writes a second `pol`, say `p2`. The planner (§5) proposes a guarded sequence that achieves this change, and the operator either accepts it (in which case we apply the sequence to the running control) or declines it. The planner is intentionally simple: it sees only `pol`-level differences whose translation to a sequence is straightforward, and falls back to a whole-tree `Replace` idiom for anything richer.
- In *imperative mode*, the operator writes both their desired `p2` and a $(\varphi; \delta)^*$ sequence intended to reach it; the sequence may include idioms from §4.2. Before running it we check the sequence at the pol level, as described next.

The two modes are not formally distinct: they produce sequences over the same substrate that discharge the same per-production obligations from §3.4. Imperative mode buys expressivity, not a different proof obligation; it admits sequences the planner might never produce, but they are still just $(\varphi; \delta)^*$ sequences.

The pol-level check on imperative sequences. The pol-level check is exactly the top edge of §4.1's iterated diagram: we fold each production's $\mathsf{den}(\delta_i)$ (§3.4) along the sequence, obtaining the intermediate pols $\mathtt{ip}_0 = \mathtt{p1}', \mathtt{ip}_1, \ldots, \mathtt{ip}_{n+1}$ with $\mathtt{ip}_{i+1} := \mathsf{den}(\delta_i)(\mathtt{ip}_i)$, where $\mathtt{p1}' = \lfloor \mathtt{C} \rfloor$ is the pol we echoed to the operator. We accept the request iff every $\mathsf{den}(\delta_i)$ is defined on $\mathtt{ip}_i$ and $\mathtt{ip}_{n+1} =_R \mathtt{p2}$. That is, the user-provided sequence actually takes `p1'` to `p2`. This is Rule 3 of $\lfloor \cdot \rfloor$ (§3.2) iterated across the sequence: each step's $\mathsf{den}(\delta_i)$ was proved to match its operational counterpart $[\![\delta_i]\!]$ per-production in §3.4, so a fold that lands at $\mathtt{ip}_{n+1} =_R \mathtt{p2}$ certifies, by

the diagram's cell-by-cell commutation, that the operational chain along the bottom edge ends in a control whose $\lfloor\cdot\rfloor$ is p2.

Each $\texttt{ip}_i$ is itself a valid pol with an explainable scheduling semantics, not a transient artifact of the proof. For instance, after the first step of Replace(path, B)'s expansion the intermediate pol holds Strict(pol@path, B) at path: a temporary strict-priority node favoring the outgoing pol@path over the incoming B, with a readable scheduling interpretation in its own right.

The "defined when" clauses on each $\texttt{den}(\delta_i)$ are the per-production preconditions listed in §3.4.1-3.4.7. Guards play no role in this check; they govern the operational timing of when each δ_i fires on the live control, and are pol-invisible (§3.4). For instance, a (empty(path); Remove(path)) step folds at the pol level exactly as if its guard were true, since den only sees δ. If the chain fails to reach p2, or some $\texttt{den}(\delta_i)$ is undefined on its intermediate pol, we reject the request. The rejection identifies the offending step: the first δ_i whose $\texttt{den}(\delta_i)$ was undefined on $\texttt{ip}_i$, or, if every step was defined, the final mismatch $\texttt{ip}_{n+1} \neq_R \texttt{p2}$.

4.4 Handling follow-up requests

This paper's transition planner engages with one reconfiguration at a time, and commits to a simple answer when the operator submits a follow-up request p3 while a p1 → p2 sequence is still mid-flight (i.e., while some guard φ_i has not yet become true): we queue p3, and the planner does not begin work on it until the in-flight sequence completes, which is to say until the live control C_z satisfies $\lfloor C_z \rfloor =_R$ p2. At that instant the planner pulls p3 from the queue, treats p2' as the new starting point, and produces a fresh $(\varphi;\delta)^*$ sequence to reach p3 exactly as in §4's main loop.

A more aggressive strategy is possible: the planner could begin work on p3 mid-flight, splicing or canceling the in-flight sequence to converge on p3 directly, sometimes at lower total cost than running p1 → p2 → p3 in series. We do not pursue this here, because the splicing analysis introduces its own correctness obligations (atomicity of the splice, what guarantees the operator has during an aborted in-flight sequence, what link the operator observes between the splice and p3) that are out of scope of this paper. A sketch of the stronger possibility lives in our discussion notes.

5 THE TRANSITION PLANNER

§4.3 referenced a *transition planner*; this section spells it out. The operator hands the runtime p1 (the presently-running policy) and p2 (their new desire); the planner's job is to propose a $(\varphi;\delta)^*$ sequence whose pol-level fold takes p1 to p2 up to $=_R$. The operator accepts or rejects the proposal; they do not author it directly.

Every sequence the planner emits is safe by §3.4 (per-production guarantees) and §4.1 (Safety of guarded sequences), regardless of how the planner assembled it. The planner aims to land at p2 by construction: each case below emits a δ or idiom whose den realizes the locally-observed pol-level difference. A misstep in the case analysis could in principle yield a sequence whose fold $\texttt{den}(\delta_n) \circ \ldots \circ \texttt{den}(\delta_1)(\texttt{p1})$ misses p2's $=_R$-class; the §4.3 pol-level check is the backstop and rejects any such sequence before commit. Optimality is a separate matter, and not guaranteed: the planner may fail to see a superior sequence that the user has in mind.

§5.1 sets up *confinement* as a metric for excellence, presents the always-available worst-case Replace fallback, and frames the planner's strategy as descending the tree to find a tight production (Add, Remove, ChangeMeta) that captures the difference locally. §5.2 catalogs the cases the planner recognizes and emits a tight sequence for. §5.3 lists deliberate non-features and candidate future work.

5.1 The Always-Available Fallback, and Confinement to do Better

To reach any p2 from any p1, the planner can issue Replace([], p2), the §4.2 idiom that expands to

```
(true ; Designate([], p2)) ;
(true ; Quiesce([0])) ;
(empty([0]) ; Undesignate([]))
```

After Designate([], p2) fires, the original p1 sits at child index 0 of the freshly-introduced Strict* wrap and the survivor p2 sits at index 1 (§3.4.5); the [0] in the next two steps targets the p1 arm. This wraps p1 and p2 into a Strict*(hi : p1, lo : p2) at the root, quiesces the p1 arm, accepts pushes into p2 and serves pops from p1, and, when p1 has drained to empty, replaces Strict*(hi : p1, lo : p2) with just p2, which then serves both pushes and pops. Nothing is dropped and the sequence is safe by §3.4, but *no part of the scheduler is left undisturbed.*

Footprint. To measure how much of the tree a sequence touches, associate each production δ with a *footprint* on the live control it acts on. Define $\texttt{footprint}(\delta, \texttt{C})$ as the set of nodes of C whose state, pifo, or z differs between C and $[\![\delta]\!](\texttt{C})$. The set is read off the per-production Preservation paragraphs of §3.4: it includes the edit site, the new or removed node where applicable, and every proper ancestor whose z is extended or restricted (e.g., the ancestors touched by Add, Quiesce, Remove, Designate). For a guarded sequence $(\varphi_0;\delta_0);\ldots;(\varphi_n;\delta_n)$, the sequence-level footprint is the union $\bigcup_i \texttt{footprint}(\delta_i, \texttt{link}_i)$. The root-level Replace([], p2) above has the maximum possible footprint: the whole tree.

The planner's strategy. The metric by which we judge the planner is *confinement*: a good emission disturbs as little of the running scheduler as possible, leaving parts of the tree that did not need to change running undisturbed. A sequence is well-confined when its footprint is small relative to the symmetric pol-level difference between p1 and p2.

The planner's goal is to recognize whether the difference between p1 and p2 is captured by a tight production from §3.3 (an Add, Remove, or ChangeMeta) and to emit that production wherever in the tree it sits, leaving everything else untouched. The planner descends through the tree as long as p1 and p2 agree at the current level, looking for a recognized shape one level down. When it finds one, it emits the matching production at that depth. Only when the descent ends without a match does the planner fall back to Replace; but by then the recursion has localized the difference to the deepest slot at which the policies disagree, so the resulting Replace wraps only that slot.

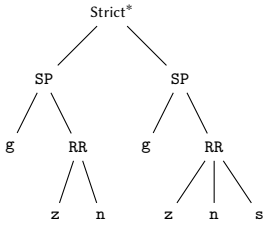
A concrete example will help. Suppose

$$\texttt{p1} = \texttt{SP}(\texttt{gmail}, \texttt{RR}(\texttt{zoom}, \texttt{netflix}))$$

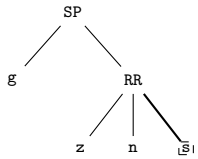
and the operator requests

$$\texttt{p2} = \texttt{SP}(\texttt{gmail}, \texttt{RR}(\texttt{zoom}, \texttt{netflix}, \texttt{spotify})).$$

The planner has two ways to realize this:



(a) Replace([], p2)



(b) Add at the RR (new arm in dashed box)

Option (a) wraps the entire running tree under a fresh Strict* and grows a freshly-compiled p2 alongside; every node sits inside the wrap, and the original p1 (with all its in-flight packets) drains as the loser. Footprint: the whole tree.

Option (b) extends the RR's arity by one and attaches a freshly compiled `spotify` leaf as the third arm; the SP and RR ancestors have only their z extended (no packet movement, no state reset), and the `gmail` leaf is left completely undisturbed. Footprint: the RR and its single ancestor.

Much of the tree is left alone under option (b). The case analysis of §5.2 is built precisely to find option (b) when it exists, and falls back to a `Replace` (at the deepest position the descent reached) only when no such tight production captures the difference.

We make no claim that the planner's choice is canonical or minimal. We claim only that whatever sequence it emits is safe (§3.4) and no worse than the root-level fallback.

5.2 What the planner emits

The planner has two parts. The top-level pass examines (p1, p2) as whole trees and one of three things happens: it finds them equal up to $=_R$ and emits `[]`, or it recognizes that p2 already lives somewhere inside p1 and therefore emits a `PruneDownTo`, or it passes the pair to a second procedure that compares two policies at one level (which we will call the *per-level comparator*). The per-level comparator either recognizes a tight production at the current level, or recurses into matching slots, or gives up and emits a slot-level `Replace`. Whatever the per-level comparator emits is lifted back to the right depth by prepending the slot indices that the recursion descended through.

The top-level pass. The pass tries three cases in order and stops at the first match.

(1) **p1 = p2.** Emit `[]`. The test is literal equality on the normalized `pol` terms. The compiler (§3.2) runs every operator-supplied policy through `Pol.normalize` before reaching the planner, so two $=_R$-equivalent policies arrive here as identical terms. The live control is already serving the requested policy and there is nothing to install.

(2) **p2 embeds in p1 at path.** Emit PruneDownTo(`path`). A structural sub-policy test walks p1 looking for a subtree equal to p2; if it finds one, `path` names where it sits. The match is unique: by §3.2's leaf-partition validity each flow in p2 may appear at most once in p1, so p1 can host p2 as a subtree in at most one position. This test runs only at the top level.

(3) **Otherwise.** Hand (p1, p2) to the per-level comparator.

The per-level comparator. The per-level comparator looks at the roots of p1 and p2. If the two roots use different constructors (for example SP and RR, or two FIFOs with different classes), it emits a slot-level `Replace` directly: the always-available §5.1 idiom restricted to the current slot. If the constructors agree, it then looks at the two child lists and compares their lengths.

Equal arity. For the constructors that carry per-arm metas (SP and WFQ), we first check for a meta-only reshuffle: if p1 and p2 hold the same multiset of arms in different positions, we emit one `ChangeMeta` per slot whose meta moved and we are done.

Otherwise, we walk the child lists slot-by-slot. A slot whose arm and meta both agree contributes nothing. A slot whose meta differs but whose arm agrees contributes a ChangeMeta(`path`, `new_meta`). A slot whose arm differs is handled by recursing to try and narrow down the difference deeper in the tree; if the meta also differs, we append a follow-on (`true`; ChangeMeta(`path`, `new_meta`)) after the inner emission.

There is one wrinkle: this slot-by-slot walk can miss a cross-slot correspondence in which an arm has moved to a different slot in p2 *and* also changed structurally. The label-overlap walk (below) catches it; when that walk finds at least one such cross-slot pairing, we use it instead of the slot-by-slot walk.

p2 longer. When p1's arms appear in order as a sub-sequence of p2's, the surplus arms in p2 are pure additions, and we emit one Add per surplus arm in ascending index order. §3.4.1's leaf-disjointness precondition holds automatically: every surplus arm's leaves are leaves of p2 not in p1, which by §3.2's leaf-partition validity are disjoint from p1's arms and from each other. At an SP or WFQ parent, each Add carries the surplus arm's meta. If a stable arm has changed metadata, we emit `ChangeMeta`s after completing the Adds.

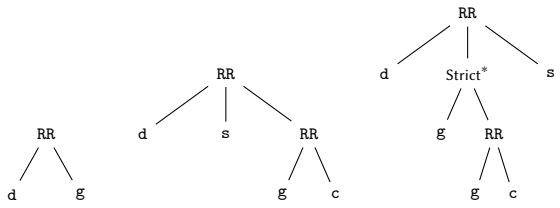
When p1's arms are *not* a sub-sequence of p2's, we try the label-overlap walk. If that fails too, we emit a slot-level `Replace`.

p1 longer. Symmetric. When p2's arms appear in order as a sub-sequence of p1's, the surplus arms in p1 are retired, and we emit one `Retire` idiom (§4.2) per surplus arm in *descending* index order. Descending order keeps the lower-index paths stable while higher-index siblings drain and disappear. When p2's arms are not a sub-sequence of p1's, we try the label-overlap walk, and emit a slot-level `Replace` if it cannot align the arms either.

The label-overlap walk. The simpler patterns above pair arms by slot position; the label-overlap walk pairs them by what they actually carry. Consider the following example.

```
p1 = RR(drive, gmail)
p2 = RR(drive, slack, RR(gmail, controller_for_gmail))
```

So the two trees are p1 and p2 below. The third panel previews what the planner installs on the substrate: not p2 literally, but a sibling-reordered representative of its $=_R$-class; we return to it below.



p1 = RR(d, g) p2 = RR(d, s, RR(g, c)) installed ($=_R$ p2)

Comparing p1 and p2: d is unchanged; g now round-robins with a control sub-channel; s is new. A slot-by-slot walk does not apply (the lengths differ), and the simpler sub-sequence check in the "p2 longer" case fails because gmail in p1 is not equal to any arm in p2. Without the walk we are about to describe, the planner would give up and emit a wholesale Replace of the entire root RR.

But the trees are not really that different. gmail in p1 and RR(gmail, gmail_control) in p2 share the class label gmail; drive is the same on both sides; slack is the only arm in p2 whose labels appear nowhere in p1. The correct reading is that drive stays, gmail has grown a sub-channel, and slack is fresh.

The label-overlap walk produces that reading. It pairs arms in p1 with arms in p2 whose class-label sets overlap (by a greedy left-to-right walk); arms in p1 with no overlapping partner retire, arms in p2 with no overlapping partner are added, and matched pairs are recursed on. The justification is that a class label identifies a unique flow across the whole policy (§3.2's leaf-partition validity), so a repeated label between an arm in p1 and an arm in p2 is evidence that the same host arm has morphed structurally, and an arm whose labels appear on no opposing arm is necessarily fresh or vanishing. For our example, the walk pairs drive with drive (no edit) and gmail with RR(gmail, gmail_control) (recurse), and treats slack as a pure add.

Our final emission is two atomic steps: an Add of slack appended at the root RR's tail, and an inner Replace at gmail's existing slot. The third panel of the figure above shows the resulting substrate state: slack sits beside the morphed gmail subtree in an order that does not match p2 literally, but §3.2's normalization invariant says sibling order at SP/RR/WFQ carries no semantics, so the re-ordering is invisible to any observer.

A few notes. The walk is greedy: when more than one alignment satisfies the label-overlap constraint, the first we find is taken, with no notion of cost (§5.3). If no shared label admits even one match, the walk fails and the caller falls back to a slot-level Replace. When the walk produces a mix of adds, retires, and matched-pair recursions, we appeal to the reorder freedom of §3.2 to keep the indexing trivial: adds are appended at the parent's tail; retires fire at their ps1 indices, in descending order so the highest-index drain doesn't shift the lower indices still pending; matched-pair recursions land at each match's position in the matches list, which is also its slot once the retires drain. Issuing the adds first is deliberate: an Add is instantaneous while a Retire waits for its subtree to drain, so the new arms start carrying traffic immediately instead of waiting out the drains.

Where the descent localizes the difference. Every recursion descends through a slot where the two trees agree or where the label-overlap walk has paired them. So by the time the per-level comparator gives up and emits a Replace at depth k, every shallower level has already matched a tight case, and the Replace lands at the deepest slot to which the planner could isolate the difference, leaving every ancestor's other subtrees running undisturbed. The slot-level Replace is the §5.1 idiom with a non-empty path: it wraps only the slot's subtree, quiesces only that subtree, and rewires only the parent edge into it. The worst case is the root-level Replace([], p2) of §5.1, reached only when the per-level comparator gives up at the root with no recursive descent.

We do not claim that the planner's emission is canonical or minimal. We claim only that it is safe (§3.4) and no worse than the root-level fallback.

5.3 What the planner does not do today

Two non-features for now; we hope to improve these in the next few meetings.

- *No cost model.* The case analysis is shape-based, not cost-weighted. A cost-aware planner could prefer different decompositions (e.g., several small Adds vs. a single root-level Replace) based on per-flow drop and delay tolerances or per-PE budgets. Presently we take the first admissible alignment without ranking competitors.
- *No imperative-mode service.* The planner serves declarative mode only. Imperative-mode authoring (§4.3) goes through the pol-level check directly, with the operator naming each δ themselves; the planner is not consulted.

6 COMPILING TO HARDWARE

6.1 The commit model and the ISA

The substrate exposes its services to the planner as an atomic-commit interface. The ISA has exactly one unit: a commit, which is a list of instructions that the substrate installs *atomically*. The entire list lands between two consecutive push/pop operations, with no intermediate state visible to the user's pop stream. This is precisely the atomicity that §3.4's *Preservation of observation* obligation demanded. A commit's *length* may depend on live control state (how many flows are admitted, how many siblings need shifting), but its *atomicity* does not.

Each δ (§3.3) lowers to one commit; that is the entire ISA-level contract. The ISA itself is unconditional: each instruction describes what to do, not what to check. The substrate trusts that its caller has issued a well-formed commit and acts accordingly. Any situation-dependent reasoning (e.g., "the subtree we are about to collapse is empty") lives at §4, in a guard φ that gated the firing of the corresponding δ, not in the substrate.

Four pieces of substrate vocabulary appear in the listing below:

- **PIFO**: one node of the PIFO-tree, addressed by an opaque id v. The substrate exposes one PIFO per tree node; how

those PIFOs are mapped onto hardware is a substrate concern (see "PE deployment" later in this section).
- **PE** (processing element): hosts one or more PIFOs and owns their per-node state and logic. A single PE can host multiple PIFOs at once; in particular, sibling PIFOs commonly cohabit a PE.
- **flow**: a traffic class.
- **index**: opaque per-PIFO handle naming one of that PIFO's children; what a PIFO uses internally to refer to a child arm.

The ISA has thirteen opcodes:

Our single-purpose opcodes also mean that the tree can drift transiently malformed *within* a commit, in ways that the commit's atomicity hides from any observer. For an instance from §6.2: `Designate`'s commit first adopts `loser` as a child of the freshly-spawned `sp_v`, then emancipates `loser` from its original parent and adopts `sp_v` in its place. Between the first adopt and the emancipate, `loser` has two parents. The atomic install means no `push` or `pop` ever sees this state, but the planner is free to issue commits whose intermediate frames would not satisfy the §3.1 well-formedness invariants.

One last piece of setup the lowerings in §6.2 will lean on: *PE deployment*. We treat the assignment of a PIFO to a PE as a deterministic function `pe(path)` of the PIFO's tree position, fixed at compile time. A typical shape is depth-by-PE: PIFOs at the same tree depth share a PE [4], so sibling PIFOs cohabit one PE by default. The planner consults `pe(path)` when it needs an `Isa_spawn`'s `pe` argument; the function's definition is a property of the target substrate and is not otherwise observable at the ISA.

6.2 Lowering Productions of δ

Each production of δ (§3.3) lowers to a list of op codes. The lowering *schema* is mechanical: each production names a fixed sequence of opcodes, and the planner reads live state at issue time to instantiate the schema's positional parameters (flow lists, chain shapes, routing indices) before dispatching the commit to the substrate.

Before working through the productions, we set up one structural convention. A switch with P output ports has P separate trees, and at the ISA we put a uniform thin wrapper around each. Every port hosts a reserved PIFO `port_root`, allocated at boot on a reserved PE, whose sole child is the actual tree root. `port_root` runs a fixed 1-arm policy. Its `Assoc` set mirrors that of the live tree's actual root; its `Map` sends each Assoc'd flow through its single index `port_step`. This `port_root` state is load-bearing for `ChangeRoot`, and it is maintained by two complementary mechanisms. On every production except `ChangeRoot`, the `walk(...)` calls that touch `Assoc`/`Map` start at `port_root` and so update its tables in lockstep with the actual root's. On `ChangeRoot`, the lowering issues no flow-table writes against `port_root` at all: the swap is exactly one `Isa_emancipate`/`Isa_adopt` pair against `port_root`, and the existing `Assoc`/`Map` state survives the swap because `port_step` is the index *name*, unbound to any particular child. The wrapper exists so that "swap the root" can be expressed as one `Isa_emancipate`/`Isa_adopt` pair against `port_root`, in the same vocabulary as any child-level edit; there is no opcode for changing the root of the tree directly.

The lowerings follow, one per production in the order of the §3.3 grammar. We fix three reading conventions for the list below. First, we identify a tree position with the PIFO that lives there. Second, when the lowering hinges on the relation between a target and its parent, we destruct δ's path as $\pi +\!\!+ [k]$ (reusing §3.4's convention: π is the parent's path, `k` is the local index by which that parent reaches the target). Third, we use shorthand for the chain walks that recur across entries. Write `flows(path)` for the set of flows admitted by some leaf under `path`. Write `chain(f)` for the unique sequence of PIFOs from `port_root` down to the leaf admitting `f`, and `internals(f)` for that sequence minus the leaf. Write `walk(op, C, f)` for issuing `op(v, f, ...)` at each $v \in C$; positional remainders such as `Isa_map`'s routing index $i_{v,f}$ are read off the live tree. When an entry's opcodes name an unmentioned parameter at π (e.g., `Isa_set_policy`'s discipline `t` or `Isa_change_arity`'s arity `n`), it is the live value at π at the time the command is issued. `Isa_spawn`'s `pe` argument is similarly elided: it is `pe(path)` for the new PIFO's position, drawn from §6.1's deployment convention.

- ChangeMeta($\pi +\!\!+ [k], m$): `Isa_set_arm_meta(`π`, k, m)`.
- Quiesce(`path`): for each $f \in$ `flows(path)`,

  `walk(Isa_deassoc, chain(`f`), `f`)`.

- Add(`path`, `arm`, `meta?`):
  - Let `n` be `path`'s current arm count; the new subtree will occupy `path`'s rightmost slot, index `n`, so existing children's indices are undisturbed.
  - Lay out the subtree `arm` in hardware. That is, for each new PIFO in the subtree compiled from `arm`, `Isa_spawn`+`Isa_adopt`+`Isa_set_policy` is issued. Per-arm `Isa_set_arm_metas` are issued where the discipline requires them.
  - `Isa_change_arity(path, n+1)` grows `path` to arity $n + 1$; where the discipline requires it, `Isa_set_arm_meta(path, n, m)` initializes the new slot's metadata from `meta?`.
  - `Isa_adopt(n, path, root_of_arm)` attaches the new subtree at index `n`.
  - Start serving traffic to the new subtree. That is, for each $f \in$ `flows(arm)`:

    `walk(Isa_assoc, chain(`f`), `f`)`;

    `walk(Isa_map, internals(`f`), `f`)`.

- Remove($\pi +\!\!+ [k]$):
  - `Isa_emancipate(k, `π`)` detaches the doomed child. π's remaining children keep their indices: each was minted at adoption time and the substrate carries it unchanged, so no sibling-shift cascade is needed.
  - `Isa_change_arity(`π`, n-1)` updates π's arity counter to match.
  - For each $f \in$ `flows(`$\pi +\!\!+ [k]$`)`,

    `walk(Isa_unmap, internals(`f`), `f`)`

    clears the doomed flows' routing through the chain above $\pi +\!\!+ [k]$ (including π itself, whose `f` entry vanishes).
  - One `Isa_gc` per PIFO in the now-detached subtree.

Opcode	Parameters	Effect
`Isa_spawn(v, pe)`	fresh PIFO id, PE id	Allocate an empty PIFO `v` on PE `pe`.
`Isa_adopt(i, p, c)`	index, parent, child	Parent `p` gains `c` as a child, reachable via index `i`.
`Isa_emancipate(i, p)`	index, parent	Inverse of `Isa_adopt`: detach `p`'s child at index `i`.
`Isa_assoc(v, f)`	PIFO, flow	`v` begins to accept packets of flow `f`.
`Isa_deassoc(v, f)`	PIFO, flow	`v` stops accepting packets of flow `f`.
`Isa_map(v, f, i)`	PIFO, flow, index	In `v`'s brain, route flow `f` to index `i`.
`Isa_unmap(v, f)`	PIFO, flow	Forget `v`'s mapping for flow `f`.
`Isa_set_policy(v, t, n)`	PIFO, policy type, initial arity	Set `v`'s policy type to `t` and its initial arity to `n`; `t` ranges over {FIFO, RoundRobin, Strict, WFQ}. Issued exactly once per PIFO, at spawn time.
`Isa_change_arity(v, n)`	PIFO, arity	Set the live `v`'s arity counter to `n`. Policy type is unchanged. `Isa_change_arity` is what tells the discipline-level logic at `v` (e.g., RR's cursor wraparound, the WFQ slot-state table's size) the active arm count; the structural attaching and detaching themselves happen through explicit `Isa_adopt`/`Isa_emancipate` calls, each keyed by the stable per-edge index the parent minted at adoption time. The substrate is free to relocate live adoptions internally as the count changes; the indices the planner holds remain valid.
`Isa_set_arm_meta(v, i, m)`	PIFO, index, metadata	Set per-arm metadata for the child reached via `i` to `m`. The payload `m` is interpreted per `v`'s policy type: a weight for RoundRobin/WFQ, a priority rank for Strict; FIFO carries none.
`Isa_designate(v, surv)`	two PIFOs	Inside an already-spawned Strict-2 super-node that has adopted both `v` and `surv` (see §6.2's `Designate` lowering), name `v` as the favored child and record `surv` as `v`'s designated successor for an eventual `Isa_undesignate`. No in-place wrapping is implied at the ISA level; the substrate may coalesce the Strict-2 super-node, `v`, and `surv` into a single physical slot when §6.2's same-PE invariant holds, but is not required to.
`Isa_gc(v)`	PIFO	Release `v`'s PE slot.
`Isa_undesignate(v)`	PIFO	Collapse the super-node $\{v \to \texttt{surv}\}$ to `surv`: the parent index that pointed at $\{v \to \texttt{surv}\}$ now points directly at `surv`, and `surv` inherits the slot's per-arm metadata.

- Designate(`path`, `survivor`): Let `loser` be the PIFO currently at `path` and `parent` be `path`'s parent. Let `k` be `path`'s last index (for `path = []`, the parent is `port_root` and `k` is `port_step`).
  - Stand up `survivor`'s subtree as in `Add`'s "lay out the subtree" step (`Isa_spawn` + `Isa_adopt` + `Isa_set_policy` per node, plus `Isa_set_arm_meta`s where the discipline requires them), with one departure: skip the `Isa_adopt` that would attach the subtree's root `surv` to a parent: the Strict-2 spawn below installs it instead.
  - Spawn and configure the Strict-2 super-node `sp_v`:
    ```
    Isa_spawn(sp_v, pe(path));
    Isa_set_policy(sp_v, Strict*, 2);
    Isa_adopt(0, sp_v, loser);
    Isa_adopt(1, sp_v, surv);
    Isa_set_arm_meta(sp_v, 0, 1.0);
    Isa_set_arm_meta(sp_v, 1, 2.0).
    ```
    `sp_v` lands on the same PE as the roots of `loser` and `surv` (see same-PE invariant below).
  - Rewire `path`'s parent edge from `loser` to `sp_v`: `Isa_emancipate(k, parent)`; `Isa_adopt(k, parent, sp_v)`. The same `k` is reused, so `parent`'s per-arm metadata for that slot is preserved across the rewire.
  - `Isa_designate(loser, surv)` marks `loser` as the favored child of the now-installed Strict-2 super-node and records `surv` as its eventual successor.
  - Wire flow routing at `sp_v` and above. At `sp_v`, for each $f \in \texttt{flows(loser)} \setminus \texttt{flows(surv)}$ issue `Isa_assoc(sp_v, f)` and `Isa_map(sp_v, f, 0)`; for each $f \in \texttt{flows(surv)}$ (whether shared with `loser` or not) issue `Isa_assoc(sp_v, f)` and `Isa_map(sp_v, f, 1)` so that the survivor takes shared classes from this commit onward. Above `sp_v`, for each $f \in \texttt{flows(surv)} \setminus \texttt{flows(path)}$: $\texttt{walk(Isa\_assoc, chain}(f), f)$; $\texttt{walk(Isa\_map, internals}(f), f)$. Flows in $\texttt{flows(surv)} \cap \texttt{flows(path)}$ need no above-`sp_v` wiring: the chain is already in place from `path`'s prior wiring. Flows in $\texttt{flows(loser)} \setminus \texttt{flows(surv)}$ (loser-only labels) keep their existing above-`sp_v` Assoc/Map state and are routed to slot 0 by `sp_v.z`, matching §3.4.5's denotational rule that loser-only labels drain on arm 0. Operator-level silencing of these flows, when the operator ultimately wants it, comes from the `Quiesce` step of the surrounding `Replace` idiom (§4), not from `Designate`.

  Same-PE invariant. The planner places `sp_v`, `loser`'s root, and `surv`'s root on the same PE (a deterministic property of the lowering: all three sit at depth $|\texttt{path}|$, and `pe(path)` returns one PE). The substrate may exploit this to coalesce the three into a single physical slot, sparing itself from moving all of `loser` down one PE level.
- Undesignate(`path`): let `sp_v` be the Strict-2 super-node head formed at `path` by the prior `Designate`, and

loser_v be its retiring arm. Isa_undesignate(loser_v); Isa_gc(sp_v) to release the super-node head (which Isa_undesignate rewires past but does not deallocate); one Isa_gc per PIFO in the now-detached subtree rooted at loser_v.

- ChangeRoot(path): let the live tree be port_root $\to a_0 \to a_1 \to \ldots \to a_m \to$ path (the §3.3 restriction makes $a_0 \ldots a_m$ a unary vine). Isa_emancipate(port_step, port_root); Isa_adopt(port_step, port_root, path); Isa_gc(a_0); Isa_gc(a_1); ...; Isa_gc(a_m).

Three items in the list deserve unpacking.

Quiesce's wide reach (root-to-path chain, plus the subtree under path) is forced by §3.2's parallel push: every node on a packet's path mints its own routing index independently, so dropping f only at some nodes would leave the rest happy to mint stray indices that would leave the tree malformed. The chain walk is what §3.4.3 calls *restricting* z *uniformly*. Quiesce does not issue Isa_unmap: silenced flows' Map entries *must outlive the silencing* so that buffered packets can be popped. Remove pairs Isa_emancipate with Isa_unmap precisely because it fires after the drain, when there is no longer any in-flight traffic to preserve.

The lowering for Designate matches §3's tree-level reading verbatim: a fresh Strict-2 PIFO sp_v is spawned, path's previous occupant becomes its favored child (rank 1.0), and the survivor becomes its second child (rank 2.0). The lowering rewires path's parent edge from loser to sp_v, reusing the same parent-side index k; the parent's per-arm metadata for the slot is therefore preserved across Designate and across the eventual Undesignate that retires the give-up. Isa_designate(loser, surv) is the ISA-level marker that names the favored child and records the eventual successor; a later Isa_undesignate swings the parent's index over to surv and releases sp_v along with the loser subtree. A substrate that recognizes the same-PE invariant on (sp_v, loser, surv) is free to coalesce the three into a single physical slot (the "super-node" optimization); the ISA does not pin this down, leaving room for substrates that prefer the literal Strict-2 form.

ChangeRoot writes no flow tables: path's Assoc and Map already describe the post-commit live tree, since collapsing the unary vine above path leaves the flows-to-leaves mapping unchanged.

6.3 Substrate portability

§3.5's argument that each δ's soundness proof survives the lowering leans on the substrate running each commit as an atomic transactional install. Two properties make this work, and they are also what any third-party substrate would need in order to host our compilation.

First, the substrate must provide the thirteen opcodes of §6.1, or compositions equivalent to them. This is a vocabulary requirement, not a semantic one: the ISA-level lowering of Designate (§6.2) is the literal Strict-2 PIFO form, which any substrate can host. A substrate that recognizes the same-PE invariant on (sp_v, loser, surv) may coalesce the three into a single physical slot, but the optimization is the substrate's prerogative, not an ISA requirement.

Second, the substrate must guarantee that no push or pop interleaves between a commit's first and last instruction. This is what hides §3.5's transiently-malformed intermediate frames. Designate's commit produces a moment in which loser has two parents (its original parent, and the freshly-spawned sp_v that has just adopted it). Remove's commit produces moments in which a node's flow-to-index map still routes flows to a just-detached child. ChangeRoot's vine collapse produces moments in which the freed vine nodes are detached but not yet released. None of these frames survive an atomic commit's install. A non-atomic substrate would expose some of them, and whether that breaks observable soundness depends on the substrate's pop-arbitration policy against an in-progress commit, a detail we do not pin down here.

The requirement is weaker than a general-purpose transaction manager. The planner knows each commit's length and shape statically, and the substrate need only honor an install-without-interleave guarantee for that fixed shape. Several productions need even less. ChangeMeta is a single instruction. A single-flow Quiesce walks one chain and modifies only Assoc; each intermediate frame is well-formed because the as-yet-unsilenced interior nodes still route via their existing Assocs.

A vPIFO-like substrate (see §2.2) would host our compilation by providing the install-without-interleave guarantee above.

[AM note for Zhiyuan: the §6.3 prose above is my best current statement of what we need from a substrate. Please confirm or push back after studying §6, then delete this note.]

6.4 Hardware Implementation

Each transition δ (§3.3) must become visible atomically in hardware: every enqueue or dequeue request should observe either the pre-δ configuration or the post-δ configuration, but never an intermediate state. This requirement is nontrivial because a single δ may lower to multiple ISA instructions (§6.2), and each instruction may update state across several physical PEs. For example, adding a flow can require changes to admission tables, flow-to-child mappings, policy metadata, and next-hop state. If these updates become visible independently, packets may be processed by a mixture of old and new configuration state, violating the preservation-of-observation requirement from §3.4.

This issue arises naturally in prior PIFO-style hardware. In the original PIFO design, a match-action pipeline maps an input packet to a PIFO entry and generates the policy parameters used by the local scheduler, while a next-hop lookup determines the next PE on the dequeue path. If the tables on one PE are updated before those on another PE, an incoming packet may be classified using the new configuration at one point in the tree but the old configuration elsewhere. Existing PIFO systems therefore rely on stop-the-world reconfiguration to avoid exposing such inconsistent states. Rio's hardware implemention focus on implementing online update: how can the hardware execute δ-level atomic updates online, without stopping packet processing?

Hardware components. To isolate the update and packet processing, Rio adds two hardware mechanisms to a conventional PIFO substrate.

First, Rio makes flow-mapping and routing-table updates atomically visible. Each PE stores its flow-to-child mapping and next-hop state in a specialized table that accepts a stream of update

instructions, and a commit signal. Updates before the signal are staged but remain invisible to packet processing. Only when the commit marker is applied does the table immidiate switch to the new version. We implement this using double buffering: update instructions modify a shadow copy of the table, and the commit marker atomically swaps the active and shadow copies. After the swap, the inactive copy is synchronized in the background to serve as the shadow for the next commit.

Second, Rio supports incremental updates to the policy match-action tables. Rio does not require arbitrary in-place replacement of a live scheduling policy. Instead, policy-table updates correspond to adding metadata for newly admitted flows (whose packets are not allowed before the udpate is finished) or removing metadata for flows that have already been drained. Because these updates do not change the interpretation of existing live packets, Rio applies them incrementally rather than through a full transactional table swap. This keeps the hardware cost low while still preserving the atomicity needed for observable scheduling behavior.

These mechanisms are independent of the underlying PIFO implementation. Our prototype uses a shift-register-based PIFO similar to the original design, but Rio only assumes the standard push-in-first-out interface. Other PIFO implementations can be substituted.

Executing δ as a transaction. Rio implements a transactional update controller that turns each lowered δ into an atomic hardware transaction. In software, the planner emits a sequence of table-update and policy-update instructions, each addressed to a specific PE and table entry. The sequence terminates with a single commit instruction and is submitted to the controller through a FIFO command queue.

In hardware, the controller executes commands in order. Ordinary update commands are routed to the target PEs and staged locally. When the controller reaches the commit instruction, it generates the hardware commit signals that make the staged updates visible. These signals are delay-aligned across PEs according to the PIFO tree topology, so that no enqueue or dequeue operation can traverse a partially updated tree. Intuitively, the controller treats the commit boundary as a wavefront: the new configuration becomes visible at different PEs at carefully chosen times, ensuring that any individual packet observes a consistent configuration along its entire path.

The controller also implements guarded transitions. As described in §4, some transitions are guarded by predicates such as the emptiness of a subtree. Rio lowers such guards to blocking commands in the controller queue. A blocking command remains at the head of the queue until its predicate becomes true; only then can the subsequent update instructions execute. FIFO execution therefore ensures that instructions depending on the guard are not issued prematurely.

Our current implementation uses a single controller queue, which provides a simple global serialization point for updates. This design is sufficient for correctness and avoids subtle ordering bugs between overlapping commits. It can, however, introduce head-of-line blocking when a guarded transition waits for a long-running drain. Supporting multiple independent queues for disjoint subtrees is a natural optimization and does not change Rio's ISA or correctness argument; we leave this extension to future work.

7 EVALUATION

8 RELATED WORK

9 CONCLUSION

REFERENCES

[1] Zhang et al., vPIFO, SIGCOMM 2024.
[2] Mohan et al., Formal Abstractions for Packet Scheduling, OOPSLA 2023.
[3] Mittal et al., LSTF, SIGCOMM 2016.
[4] Sivaraman et al., PIFO, SIGCOMM 2016.